



PES
UNIVERSITY

CELEBRATING 50 YEARS

Mathematics for Computer Science Engineers

Unit - 4 Introduction to Optimization

Chandravva Hebbi

Computer Science and Engineering

Mathematics for Computer Science Engineers

Introduction to Optimization



What is Optimization?

- **Definition:** Optimization is the process of finding the best possible solution to a problem within given constraints.
- **Objective:** It involves maximizing or minimizing an objective function, such as cost, time, efficiency, or resource utilization.

Optimization in Computer Science

- **Critical Role:** Optimization helps improve algorithms, allocate network resources, and maximize software performance.
- **Example Problem — Network Routing:** The goal is to determine the most efficient path for data across a network, minimizing latency and maximizing bandwidth. Constraints such as bandwidth limits, congestion, and maintenance costs are considered for optimal data flow.
- **Industry Application:** Cisco Systems, a leader in networking, relies heavily on network routing optimization to ensure fast and reliable data transfer between offices.

Mathematics for Computer Science Engineers

Introduction to Optimization



Why Optimization?

- **Efficient Resource Utilization:** Ensures best use of available resources (time, money, materials).
- **Improved Decision-Making:** Helps select the most effective solution from alternatives.
- **Maximizing Performance:** Increases speed, accuracy, output, or other performance metrics.
- **Cost Reduction:** Minimizes waste, reducing costs across processes and industries.
- **Handling Complex Problems:** Offers structured approaches for real-world challenges with multiple constraints.
- **Scalability:** Allows solutions to scale efficiently with complexity or size.

Real-world applications of optimization:

Category	Example	Company	Purpose of Use
Constrained Optimization	Resource Allocation Problem	Google	Optimizing server resources under given budget and performance constraints
Unconstrained Optimization	Neural Network Training	Facebook	Minimizing prediction error by adjusting weights without constraints

Mathematics for Computer Science Engineers

Components of an Optimization Problem



Components of an Optimization Problem

- **Decision Variables**
- **Objective Function**
- **Constraints**
- **Feasible and Optimal Region**
- **Methods to solve the problem**

1. Decision Variables

- Decision variables are those that can be **controlled or adjusted** to achieve the optimal value of the objective function.
- These are the **unknown quantities** that need to be determined.
- *Example:* In house price prediction, the slope (β_1) indicates price change per unit of square footage, and the intercept (β_0) gives the price when the square footage is zero.

2. Objective Function

- The objective function is what needs to be **maximized or minimized**.
- Optimization seeks input values that maximize or minimize this function; it quantifies the goal of the process.
- *Example:* For predicting house prices by square footage, the objective function can be the squared difference between the predicted and actual prices for all houses.

3. Constraints

- Constraints are **conditions or limitations** imposed on decision variables.
- They can be **equality constraints** (variables must satisfy an equation) or **inequality constraints** (variables must lie within a range).
- *Example:* A constraint may require that the slope (β_1) is positive, enforcing that house prices increase with more square footage.

4. Feasible Region

- The feasible region is the **set of all possible values** for decision variables that satisfy every constraint (including non-negativity).
- The optimal solution must be within this region.
- *Example:* With no constraints, all possible slopes and intercepts are feasible for the regression line.

Mathematics for Computer Science Engineers

Components of an Optimization Problem



5. Feasible and Optimal Solution

- **Feasible Solution:** Any solution that meets all the constraints of the optimization problem. It may not necessarily be the best (optimal) outcome for the objective function.
- **Optimal Solution:** The solution among all feasible solutions that **maximizes** or **minimizes** the objective function, reflecting the best achievable outcome within the constraints.

6. Methods to Solve the Above Optimization Problem

1. Graphical Method (for 2 variable input)
2. Simplex Method (For more than 3 variable input)
3. Branch and Bound (For discrete input)
4. Lagrangian (For non-linear optimization problems with equality constraints)
5. KKT (For non-linear optimization problems with inequality constraints)
6. Gradient Descent
7. Evolutionary optimization Technique : Genetic Algorithm

Mathematics for Computer Science Engineers

Model Formulation



Mathematical Formulation of Optimization Problems

General form:

Optimize $f(x)$, Subjected to $g_i(x) \leq 0, \quad i = 1, \dots, m, \quad h_j(x) = 0, \quad j = 1, \dots, p, \quad x \in R^n$

Where,

$f(x)$: Objective function

$g_i(x)$: Inequality constraints

$h_j(x)$: Equality constraint

x : Decision variables

Main Components:

Objective Function: $f(x)$, to be maximized or minimized

Constraints:

- Inequality: $g(x) \leq 0$
- Equality: $h(x)=0$

Decision Variables: $x = (x_1, x_2, \dots, x_n)$

Feasible Region: R^n , set of all x that satisfy the constraints

Example: House Price Prediction

Objective: Predict the price of a house based on its size (square feet) and number of bedrooms, while minimizing prediction error.

Decision Variables:

- β_0 : Intercept (base price when size & bedrooms are zero)
- β_1 : Coefficient for size (effect on price per square foot)
- β_2 : Coefficient for number of bedrooms (effect on price per bedroom)
- **Objective Function: Mathematical Expression**

To minimize the total prediction error (sum of squared errors) across training data points, the objective function is:

$$\text{Minimize } E = \sum_{i=1}^n (P_i - (\beta_0 + \beta_1 S_i + \beta_2 B_i))^2 \quad \text{Subject to: } \beta_0 \geq 0, \beta_1 \geq 0, \beta_2 \geq 0$$

Where:

- E : total error (sum of squared errors)
 - S_i : size of house i (square feet)
 - P_i : actual price of house i
 - B_i : number of bedrooms in house i
- This is a non linear function(sum of squared errors)**

A factory produces **chairs** and **tables**. Each chair gives a profit of ₹200 and each table ₹500. Producing one chair requires 4 hours of labour and 2 kg of wood, while a table requires 6 hours and 3 kg of wood. The factory has at most 60 hours of labour and 30 kg of wood available per week.

Formulation:

Let

x_1 = number of chairs,

x_2 = number of tables.

Objective:

Maximize profit

$$Z = 200x_1 + 500x_2$$

Constraints:

$$4x_1 + 6x_2 \leq 60$$

$$2x_1 + 3x_2 \leq 30$$

$$x_1, x_2 \geq 0$$

A company has 3 machines that can produce two products, A and B. Machine I can run 100 hours, Machine II 80 hours, Machine III 90 hours. Each unit of A requires 2, 1, and 1 hours on machines I, II, and III respectively. Each unit of B requires 1, 2, and 3 hours. Profit per unit of A and B are ₹40 and ₹50 respectively.

Formulation:

Let x_1 = units of A, x_2 = units of B.

Objective:

Maximize $Z = 40x_1 + 50x_2$

Constraints:

$$2x_1 + x_2 \leq 100$$

$$x_1 + 2x_2 \leq 80$$

$$x_1 + 3x_2 \leq 90$$

Mathematics for Computer Science Engineers

Model formulation example



A person wants to meet daily nutritional requirements:

- At least 60g of protein
- At least 40g of carbohydrates

They can choose from two foods:

Food 1: 30g protein, 10g carbs per unit, costs ₹20

Food 2: 10g protein, 20g carbs per unit, costs ₹10

Formulation:

Let x_1, x_2 = units of food 1 and food 2.

Objective:

Minimize cost $Z = 20x_1 + 10x_2$

Constraint:

$$30x_1 + 10x_2 \geq 60$$

$$10x_1 + 20x_2 \geq 40$$

$$x_1, x_2 \geq 0$$

A cloud service provider has a server that can run two applications: **App A** and **App B**. The provider has the following data:

Profit:

- App A generates a profit of \$50 per instance
- App B generates a profit of \$40 per instance

Resource Usage:

- Each instance of App A requires 3 units of CPU and 2 units of memory
- Each instance of App B requires 2 units of CPU and 4 units of memory

Resource Availability:

- The total available CPU is 120 units
- The total available memory is 80 units

Define Decision Variables:

- x_1 : Number of instances of App A to deploy
- x_2 : Number of instances of App B to deploy
- **Objective Function:** Maximize the total profit Z:
 - $Z = 50x_1 + 40x_2$
- **Constraints:** Consider the resource limitations for CPU and memory:
- **CPU Constraint:** $3x_1 + 2x_2 \leq 120$
- **Memory Constraint:** $2x_1 + 4x_2 \leq 80$.
- **Non-negativity Constraints:** $x_1 \geq 0$; $x_2 \geq 0$

1.

Scenario: A company manufactures two products, Product A and Product B. The objective is to maximize profit, where Product A gives a profit of \$50 per unit, and Product B gives a profit of \$30 per unit. However, there are constraints on resources like labour hours and raw materials.

Objective Function: Maximize output $Z = 50x_1 + 30x_2$

Constraints: Labor: $2x_1 + x_2 \leq 40$

Materials: $3x_1 + 2x_2 \leq 60$

Non-negativity: $x_1, x_2 \geq 0$

2.

Scenario: Task scheduling problem where tasks need to be assigned to processors, and the objective is to minimize total computation time.

Objective Function: Minimize computation time $Z = t_1x_1 + t_2x_2$

Constraints: Processor availability: $x_1 + x_2 \leq A$

Task dependencies: Constraints on the sequence of task execution.

Mathematics for Computer Science Engineers

Model Formation



3.

Scenario: Optimizing the production of two types of enzymes, Enzyme A and Enzyme B, in a biotech lab to maximize output. Each enzyme requires different amounts of chemical inputs and time in a bioreactor.

Objective Function: Maximize profit $Z = p_1x_1 + p_2x_2$ (p_1 and p_2 are profit associated with each enzyme)

Constraints: Chemical Input: $c_1x_1 + c_2x_2 \leq R$ (resource constraint)

Reactor Time: $t_1x_1 + t_2x_2 \leq T$ (time constraint)

4.

Scenario: An electric grid optimization problem where we want to maximize power supply while minimizing costs. Different power plants have different production costs and capacities.

Objective Function: Minimize cost $Z = c_1x_1 + c_2x_2$ (c_1 and c_2 are cost associated with each enzyme)

Constraints: Power Demand: $x_1 + x_2 \geq D$

Capacity Limits: $x_1 \leq C_1, x_2 \leq C_2$

Constrained Optimization

- Linear Optimization
 - Linear Programming Problems (LPP)
 - Integer Programming
 - Dynamic Programming
- Non-Linear Optimization
 - Non-Linear Integer Programming
 - Non-Linear Dynamic Programming

Unconstrained Optimization

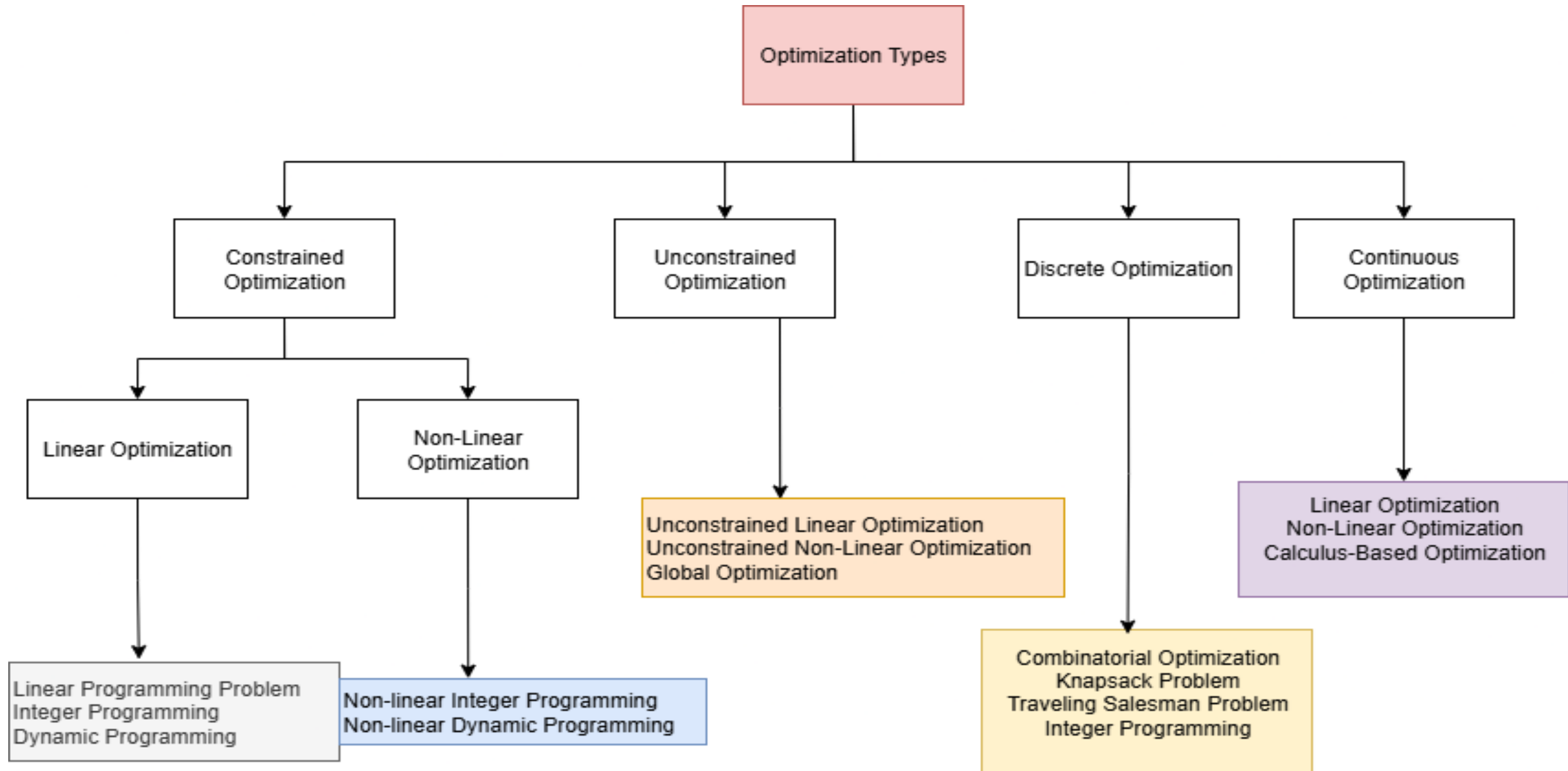
- Unconstrained Linear Optimization
- Unconstrained Nonlinear Optimization
- Global Optimization

Discrete Optimization

- Combinatorial Optimization
- Knapsack Problem
- Traveling Salesman Problem
- Integer Programming

Continuous Optimization

- Linear Optimization
- Non-Linear Optimization
- Calculus-Based Optimization



1. Engineering Optimization Theory and Practice, Fifth Edition, Singiresu S. Rao, 2020.



THANK YOU

Chandravva Hebbi

Computer Science and Engineering



PES
UNIVERSITY

CELEBRATING 50 YEARS

Mathematics for Computer Science Engineers

Unit - 4 Constrained Optimization

Chandravva Hebbi

Computer Science and Engineering

Mathematics for Computer Science Engineers

Graphical Solution Method



Linear Programming Problem (LPP) Context:

- A decision maker aims to:
 - Maximize (e.g., revenue, profit) or
 - Minimize (e.g., costs) an objective function Z .
- Z depends on decision variables.
- Constraints impose limitations on variables using inequalities ($\leq$, $=$, $\geq$).
- Variables also typically have non-negativity restrictions (≥ 0).

Methods to Solve Linear Programming Problem

1. Graphical Method
2. Simplex Method

Graphical Solution

Graphical solution is limited to linear programming models containing only two decision variables (can be used with three variables but only with great difficulty).

- Graphical methods provide visualization of how a solution for a linear programming problem is obtained.
- Graphical method: Extreme-point evaluation Method (Corner point method).

Mathematics for Computer Science Engineers

Graphical Solution Method



Graphical Solution Steps

- Step I: Convert each inequality as equation.
- Step II: Plot each equation on the graph.
- Step III: Shade the 'Feasible Region'. Highlight the common Feasible region.
Feasible Region: Set of all possible solutions.
- Step IV: Compute the coordinates of the corner points (of the feasible region). These corner points will represent the 'Feasible Solution'.
Feasible Solution: If it satisfies all the constraints and non-negativity restrictions.
- Step V: Substitute the coordinates of the corner points into the objective function to see which gives the Optimal Value. That will be the 'Optimal Solution'.
Optimal Solution: If it optimizes (maximizes or minimizes) the objective function.
Unbounded Solution: If the value of the objective function can be increased or decreased indefinitely, such solutions are called Unbounded solution.
Inconsistent Solution: It means the solution of problem does not exist. This is possible when there is no common feasible region.

Mathematics for Computer Science Engineers

Graphical Method-Maximization Example-1

How many bowls and mugs should be produced to maximize profits given labour and materials constraints?

Decision Variables:

x_1 = number of bowls,

x_2 = number of mugs.

Objective function:

Maximize $Z = 40x_1 + 50x_2$

Constraints:

$x_1 + 2x_2 \leq 40$ (labour hours constraint)

$4x_1 + 3x_2 \leq 120$ (clay constraint)

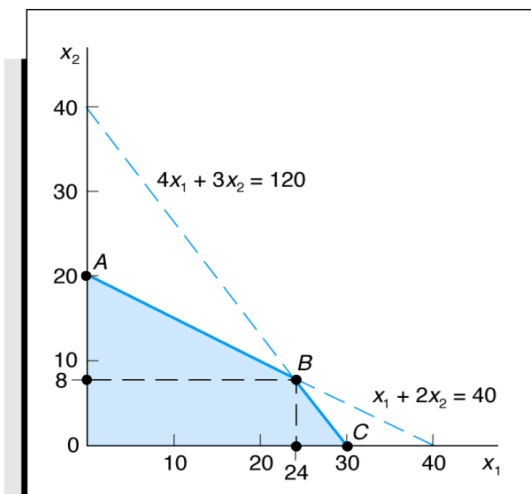
$x_1 \geq 0, x_2 \geq 0$ (non-negativity)

Solution: Max at $(x_1, x_2) = (24, 8)$

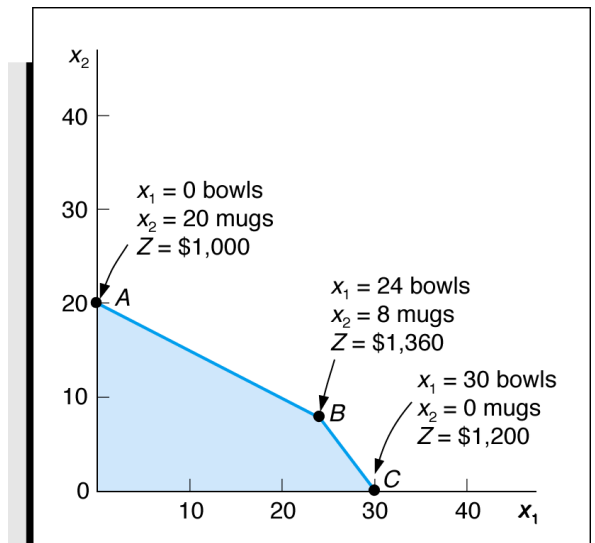
$Z_{\max} = 1360$

Resource	Bowls (Units)	Mugs (Units)	Availability
Labor (Hr./Unit)	1	2	40 Hours per day
Clay (Lb./Unit)	4	3	120 pounds
Profit (\$/Unit)	40	50	Objective: Maximize Z

Maximize $Z = \$40x_1 + \$50x_2$
 subject to:
 $1x_1 + 2x_2 \leq 40$
 $4x_1 + 3x_2 \leq 120$
 $x_1, x_2 \geq 0$



Optimal Solution Coordinates



A company manufactures two types of electronic devices: Device 1 and Device 2. The profit earned from selling one unit of Device 1 is \$7, and from Device 2 is \$10. The company wants to decide how many units of each device to produce in order to maximize its total profit.

However, the production process is limited by several constraints:

1. Fabrication Time Constraint: Each unit of Device 1 requires 3 hours of fabrication, and each unit of Device 2 requires 2 hours. The total available fabrication time per production period is 36 hours.
2. Assembly Time Constraint: Each unit of Device 1 requires 2 hours of assembly, and each unit of Device 2 requires 4 hours. The total available assembly time per production period is 40 hours.
3. IC Chips Constraint: Each unit of Device 1 consumes 10 integrated circuit (IC) chips. The company has only 100 IC chips available per production period. (Device 2 does not use these chips or has a separate supply)
4. Non-negativity Constraints: The company cannot produce negative quantities, so the number of units produced for both devices must be zero or more.

Mathematics for Computer Science Engineers

Graphical Method-Maximization Example-2

Maximize profit: $P = 7X_1 + 10X_2$

Subject to:

- Fabrication Time: $3X_1 + 2X_2 \leq 36$
- Assembly Time: $2X_1 + 4X_2 \leq 40$
- IC Chips: $10X_1 \leq 100$
- Non-negativity: $X_1, X_2 \geq 0$

Solution: Check corner (feasible) points:

$(0,0) \rightarrow P=0$

$(0,10) \rightarrow P = 7 \times 0 + 10 \times 10 = 100$

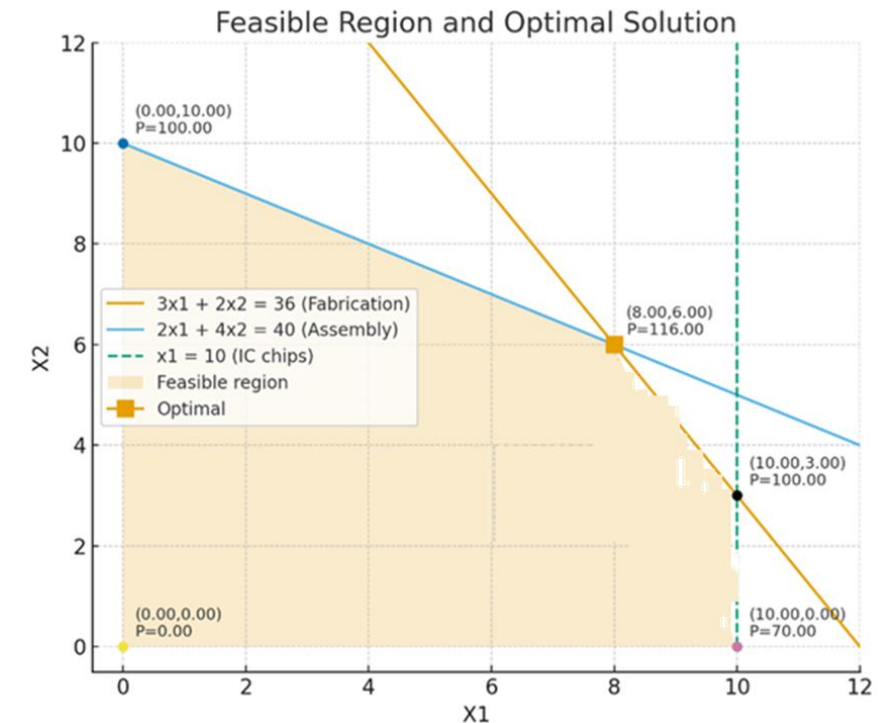
$(8,6) \rightarrow P = 7 \times 8 + 10 \times 6 = 56 + 60 = 116$

$(10,0) \rightarrow P = 7 \times 10 + 10 \times 0 = 70$

$(10,3) \rightarrow P = 7 \times 10 + 10 \times 3 = 70 + 30 = 100$

The maximum occurs at the intersection $(X_1, X_2) = (8,6)$

Maximum profit: $P_{\max} = 116$.



Mathematics for Computer Science Engineers

Graphical Method-Minimization Example-1



A company produces two products, Product 1 and Product 2. The cost to produce one unit of Product 1 is \$6, and for Product 2 is \$3. The company wants to minimize the total production cost while meeting certain minimum requirements.

The production is subject to the following constraints:

1. The total production must satisfy a minimum resource requirement such that for every unit of Product 1 produced, 2 units of a particular resource are used, and for every unit of Product 2 produced, 4 units are used. The company needs at least 16 units of this resource overall.
2. Another resource constraint requires that for every unit of Product 1, 4 other units of a different resource are needed, and for every unit of Product 2, 3 units of that resource are required. The total of this resource must be at least 24 units.
3. The production quantities for both products cannot be negative.

Mathematics for Computer Science Engineers

Graphical Method-Minimization Example-1

Minimize $Z = 6x_1 + 3x_2$
subject to: $2x_1 + 4x_2 \geq 16$
 $4x_1 + 3x_2 \geq 24$
 $x_1, x_2 \geq 0$

Solution: Check corner (feasible) points:

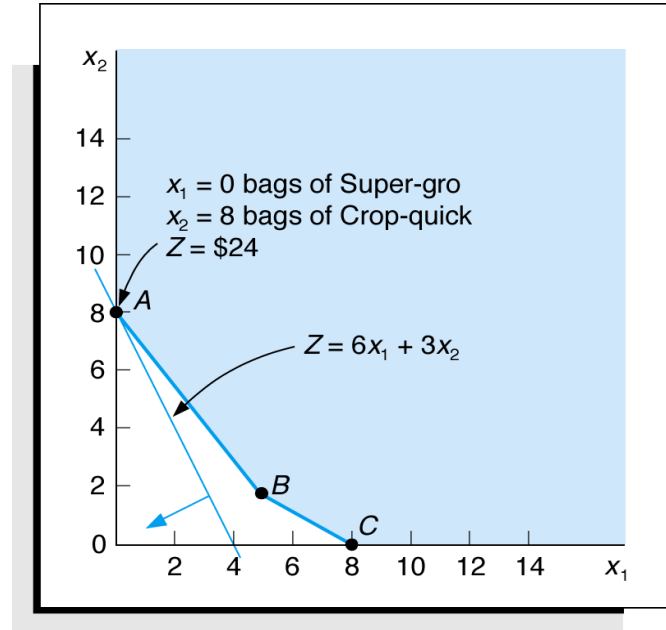
$(0,8) \rightarrow P=24$

$(8,0) \rightarrow P = 48$

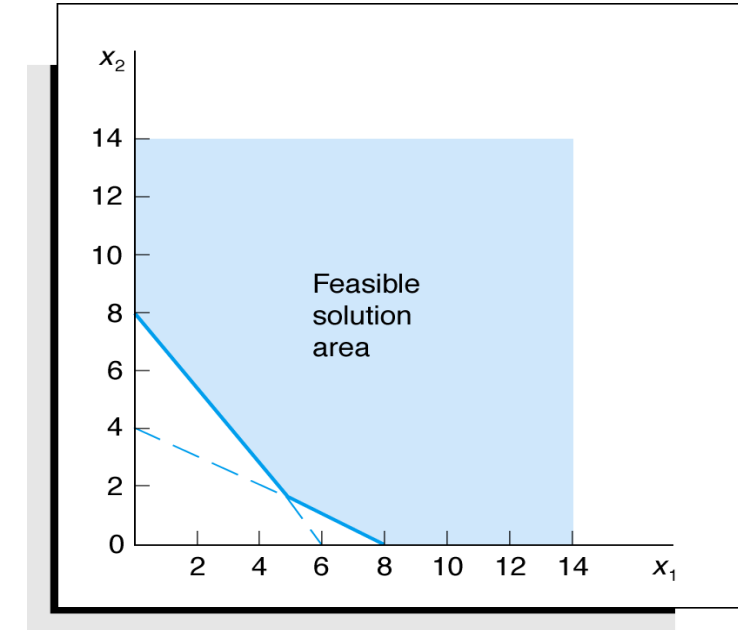
$(4.8,1.6) \rightarrow P = 33.6$

The min occurs at the intersection $(X_1, X_2) = (0,8)$

Maximum profit: $P_{\max} = 24$.



Optimum Solution Point



Feasible Solution Area

Mathematics for Computer Science Engineers

Graphical Method-Unbounded Example



A fertilizer company produces two products — **Super-Gro** and **Crop-Quick**. Each product contributes to the company’s **profit** as follows:

- Each bag of **Super-Gro** gives a profit of ₹6.
- Each bag of **Crop-Quick** gives a profit of ₹3.

Production of these fertilizers requires two key nutrients — **Nitrogen** and **Phosphate** — which are available in limited quantities. Each Super-Gro bag requires **3 units of nitrogen** and **1 unit of phosphate**. Each Crop-Quick bag requires **1 unit of nitrogen** and **1 unit of phosphate**.

The company has a supply of **at least 8 units of nitrogen** and **at least 5 units of phosphate** available for production. The management wants to determine how many bags of each fertilizer to produce in order to **maximize total profit**.

Brand	Nitrogen (lb/bag)	Phosphate (lb/bag)
Super-gro	2	4
Crop-quick	4	3

Mathematics for Computer Science Engineers

Graphical Method-Unbounded Example

Decision Variables

- x_1 : Number of Super-gro bags to produce.
- x_2 : Number of Crop-quick bags to produce.

Objective Function

Maximize total profit:

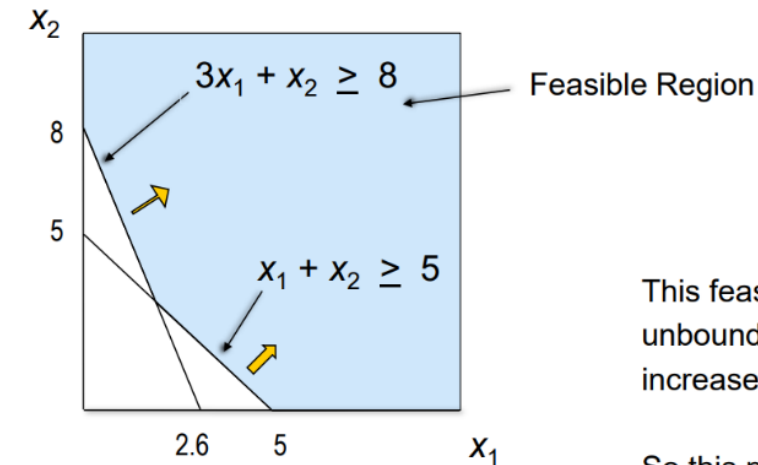
$$Z = 6x_1 + 3x_2$$

Where:

- $6x_1$: Cost of Super-gro bags,
- $3x_2$: Cost of Crop-quick bags.

Constraints

- **Nitrogen:** $3x_1 + x_2 \geq 8$
- **Phosphate:** $x_1 + x_2 \geq 5$
- **Non-negativity:** $x_1 \geq 0, x_2 \geq 0$



This feasible region is unbounded, hence z can be increased infinitely.

So this problem is having **Unbounded Solution.**

In this example, common feasible region does not exist and hence the problem is not having an optimal solution. This is the case of infeasible solution.

Mathematics for Computer Science Engineers

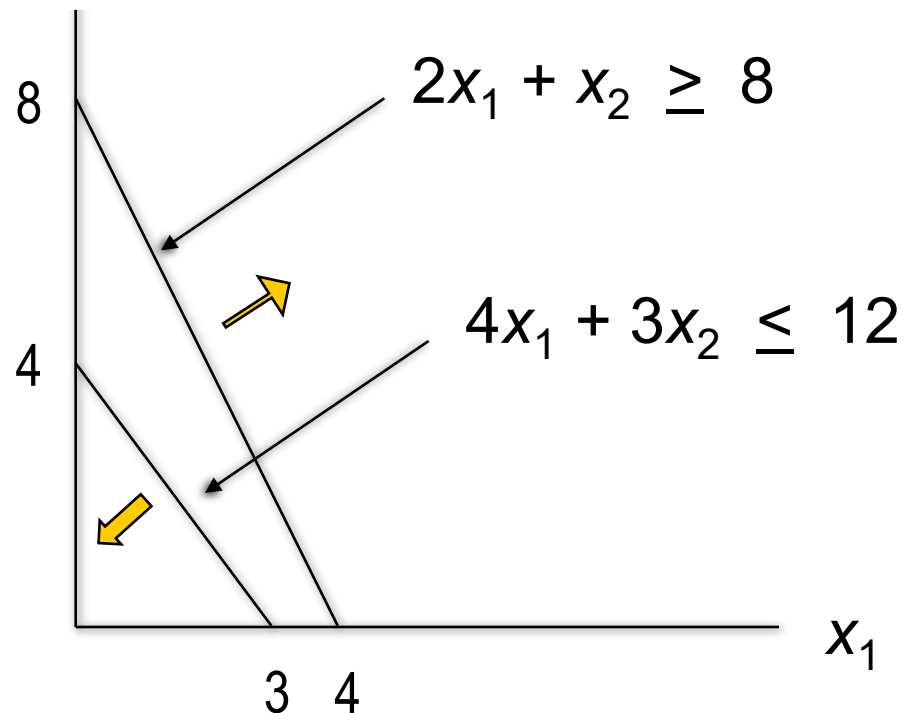
Graphical Method-Infeasible Solution Example

Maximize $Z = 2x_1 + 6x_2$

Constraints: $4x_1 + 3x_2 \leq 12$

$2x_1 + x_2 \geq 8$

$x_1, x_2 \geq 0$



In this example, common feasible region does not exist and hence the problem is not having a optimal solution.

This is the case of **infeasible solution**.

Note: The graphical approach can be used for two-variable LP problems Unfortunately, most real-life LPs problems require a method to find optimal solutions capable of dealing with several variables: the simplex algorithm

Simplex Method for solving LPP

Mathematics for Computer Science Engineers

Simplex Method Example 1



Example Problem

A forestry company manages two types of tree plantations: **pine** and **eucalypt**.

- Each hectare of **pine** yields a profit of **\$90**.
- Each hectare of **eucalypt** yields a profit of **\$120**.

However, there are several constraints on land and resources:

1. The company has at most **40 hectares** of land suitable for planting pine.
2. It has at most **50 hectares** suitable for eucalypt.
3. Due to limitations in **labour and water availability**, the total resource usage must not exceed a limit.
 - Each hectare of pine requires **2 resource units**.
 - Each hectare of eucalypt requires **3 resource units**.
 - The total available resource units are **180**.

The company wants to determine how many hectares of pine (x_1) and eucalypt (x_2) to plant in order to **maximize total profit**.

Mathematics for Computer Science Engineers

Simplex Method Example 1



Model formation

Maximize objective function: $Z = 90x_1 + 120x_2$

where:

- x_1 : hectares of pine, x_2 : hectares of eucalypt.

Subject to constraints:

- $x_1 \leq 40$
- $x_2 \leq 50$
- $2x_1 + 3x_2 \leq 180$
- $x_1, x_2 \geq 0$

Mathematics for Computer Science Engineers

Simplex Method Formulation

The **Simplex algorithm** is an algebraic procedure to solve LP problems based on **geometric concepts** that must be translated into algebraic language to allow solving systems of equations.

1st - transform all inequalities into equalities by introducing one additional variable to each constraint (the slack variables: S_1 , S_2 , S_3).

2nd - transform the objective function into an additional constraint

- $x_1 + S_1 = 40$
- $x_2 + S_2 = 50$
- $2x_1 + 3x_2 + S_3 = 180$

3rd - build the Simplex tabular form where only the essential information is recorded

basic var.	coefficients of:						right side
	Z	x1	x2	S1	S2	S3	
Z	1	-90	-120	0	0	0	0
S1	0	1	0	1	0	0	40
S2	0	0	1	0	1	0	50
S3	0	2	3	0	0	1	180

Non-basic variables

Basic variables

Each basic feasible solution has **basic** or **non-basic** variables

- **non-basic variables** are set to ZERO
- **basic variables** are directly obtained from the table

initialize the procedure setting $x_1 = x_2 = 0$

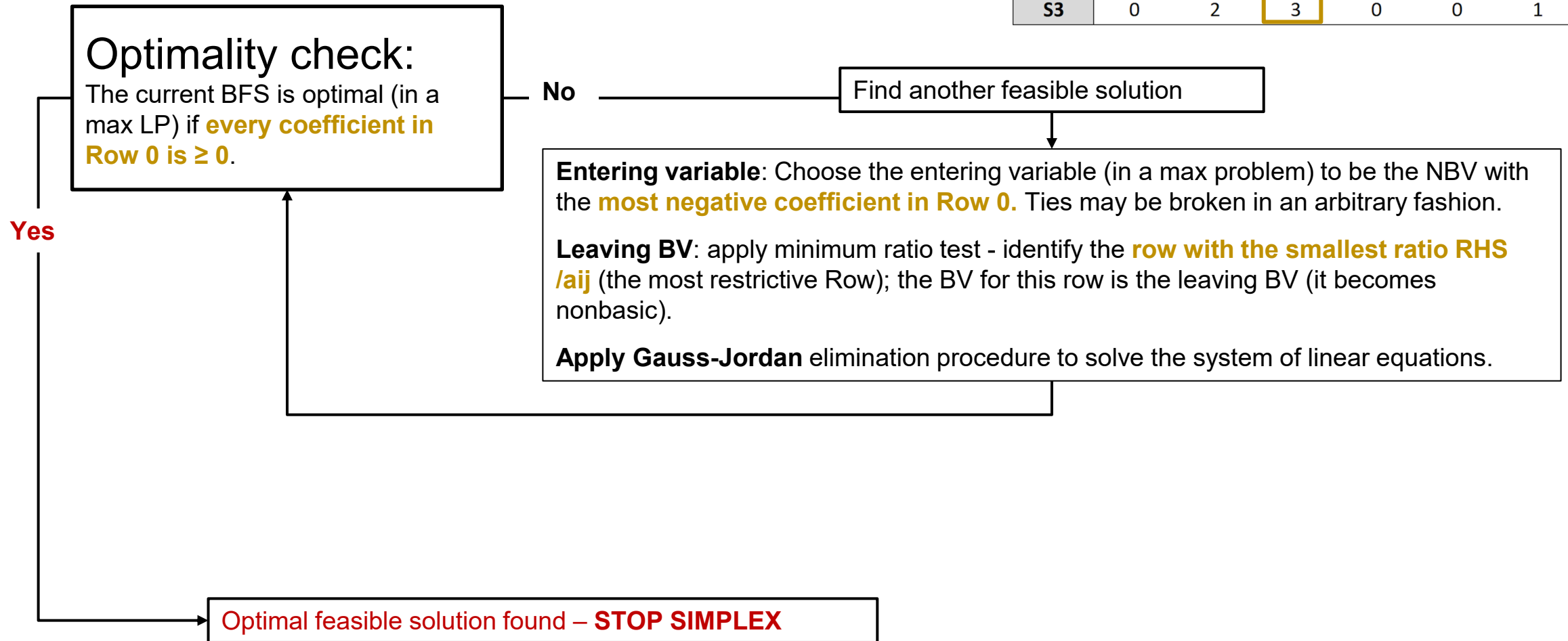
$$(X_1, X_2, S_1, S_2, S_3) = (0, 0, 40, 50, 180)$$

Mathematics for Computer Science Engineers

Simplex Method Procedure

basic var.	coefficients of:						right side
	Z	x1	x2	S1	S2	S3	
Z	1	-90	-120	0	0	0	0
S1	0	1	0	1	0	0	40
S2	0	0	1	0	1	0	50
S3	0	2	3	0	0	1	180

Bring the LP problem to the standard form



Mathematics for Computer Science Engineers

Simplex Method Example 1



Objective as equation: $Z - 90x_1 - 120x_2 = 0$

Solution:

Initial Simplex Table (canonical form)

Basic Var.	Z	x_1	x_2	S_1	S_2	S_3	RHS
Z	1	-90	-120	0	0	0	0
S1	0	1	0	1	0	0	40
S2	0	0	1	0	1	0	50
S3	0	2	3	0	0	1	180

Interpretation: basic variables are S_1, S_2, S_3 (their RHS values are the initial basic solution). Objective row shows negative coefficients under x_1, x_2 so we can improve Z.

Mathematics for Computer Science Engineers

Simplex Method Example 1



Step 1 — Choose entering variable and leaving variable

- **Entering variable:** pick the most negative coefficient in the Z-row. Here -120 (under x_2) is more negative than -90 , so x_2 enters.
- **Leaving variable:** do the ratio test for rows where x_2 coefficient > 0 :
 - S_1 row: coefficient $0 \rightarrow$ not eligible
 - S_2 row: coefficient $1 \rightarrow$ ratio = RHS / coeff = $50/1 = 50$
 - S_3 row: coefficient $3 \rightarrow$ ratio = $180/3=60$
 - **Minimum ratio = 50** $\rightarrow S_2$ leaves. Pivot element = 1 (row S_2 , col x_2).

Row operations (pivot at row S_2 , column x_2 — pivot = 1)

- Pivot row (S_2) already has 1 in x_2 , so no scaling required.
- Eliminate x_2 in other rows:

$$R_Z := R_Z + 120 * R_{S_2}$$

$$R_{S_1} := R_{S_1} \text{ (no change, } x_2 \text{ coeff} = 0)$$

$$R_{S_3} := R_{S_3} - 3 * R_{S_2}$$

Mathematics for Computer Science Engineers

Simplex Method Example 1

Compute the numbers:

R_z new:

$$\begin{aligned} \text{Z-row new} &= [1, -90 + 120 \times 0, -12 \times 0 + 120 \times 1, 0 + 120 \times 0, 0 + 120 \times 1, 0 + 120 \times 0 \mid 0 + 120 \times 50] \\ &= [1, -90, 0, 0, 120, 0 \mid 6000] \end{aligned}$$

R_{S3} new = $R_{S3} - 3R_{S2}$:

$$\begin{aligned} &= [0, 2 - 3 \times 0, 3 - 3 \times 1, 0 - 3 \times 0, 0 - 3 \times 1, 1 - 3 \times 0 \mid 180 - 3 \times 50] \\ &= [0, 2, 0, 0, -3, 1 \mid 30] \end{aligned}$$

So, the table becomes:

Basic Var.	Z	x_1	x_2	S_1	S_2	S_3	RHS
Z	1	-90	0	0	120	0	6000
S1	0	1	0	1	0	0	40
x2	0	0	1	0	1	0	50
S3	0	2	0	0	-3	1	30

Remarks: x_2 replaced S_2 in basis (row label now x_2). Z improved from 0 to Z=6000 (RHS of Z-row).

Mathematics for Computer Science Engineers

Simplex Method Example 1



Step 2 — Next pivot

- Look at Z-row: coefficients under decision variables: x_1 column = -90 (still negative) and x_2 column = 0 . So x_1 should enter next.
- Ratio test for x_1 (rows where coefficient > 0):
 - S_1 row: coeff of $x_1 = 1 \rightarrow \text{ratio} = 40/1=40$
 - x_2 row: coeff = $0 \rightarrow$ not eligible
 - S_3 row: coeff = $2 \rightarrow \text{ratio} = 30/2=15$
 - **Minimum ratio = 15** $\rightarrow S_3$ leaves. Pivot element = 2 (row S_3 , col x_1).

Row operations (pivot at S_3 row, x_1 column)

1. **Scale pivot row** to make pivot = 1:

$$R_{S_3} := R_{S_3}/2$$

So new pivot row (becomes x_1 row after pivot):

$$R_{x_1} = [0, 1, 0, 0, -3/2, 1/2 \mid 15]$$

(i.e. $S_3/2$)

2. Eliminate x_1 from other rows:

$$R_Z := R_Z + 90 * R_{x1}$$

$$R_{S1} := R_{S1} - 1 * R_{x1}$$

$$R_{x2} := R_{x2} \text{ (} x_1 \text{ coeff is zero here, no change)}$$

Compute the numbers exactly:

· New R_Z :

$$\text{previous } R_Z = [1, -90, 0, 0, 120, 0 \mid 6000]$$

$$\text{add } 90 \times R_{x1} = [0, 90, 0, 0, -135, 45 \mid 1350]$$

$$\rightarrow R_Z \text{ new} = [1, 0, 0, 0, -15, 45 \mid 7350]$$

· New R_{S1} :

$$\text{previous } R_{S1} = [0, 1, 0, 1, 0, 0 \mid 40]$$

$$\text{subtract } R_{x1} = [0, 1, 0, 0, -3/2, 1/2 \mid 15]$$

$$\rightarrow R_{S1} \text{ new} = [0, 0, 0, 1, 3/2, -1/2 \mid 25]$$

$$\cdot R_{x2} \text{ remains } [0, 0, 1, 0, 1, 0 \mid 50].$$

Mathematics for Computer Science Engineers

Simplex Method Example 1

So, the table after the second pivot is:

Basic Var.	Z	x_1	x_2	S_1	S_2	S_3	RHS
Z	1	0	0	0	-15	45	7350
S_1	0	0	0	1	$3/2$	$-1/2$	25
x_2	0	0	1	0	1	0	50
x_1	0	1	0	0	$-3/2$	$1/2$	15

(We renamed the scaled S_3 row to x_1 since x_1 is now basic there.)

Iteration 3

- Check for negative coefficients in Z-row: The rule for the simplex method is to continue as long as there is a negative coefficient in the objective row, for any non-basic variable.
- Here, S_2 (a slack variable) has a negative coefficient (-15). If we want to drive all slack variables to 0 in the objective row for the highest Z, and strictly apply the simplex method, S_2 should enter the basis.

Mathematics for Computer Science Engineers

Simplex Method Example 1



Step 3 (Next Pivot)

1. Entering variable: S_2 (has -15 in Z-row)

2. Leaving variable: Do ratio test where S_2 coefficient > 0 :

- S_1 row: Divide by $3/2 \rightarrow \text{Ratio} = 25 / (3/2) = 16.666\dots$
- x_2 row: Divide by $1 \rightarrow \text{Ratio} = 50/1 = 50$
- x_1 row: $-3/2$ (negative, not eligible)

Minimum ratio: 16.67 (row S_1).

So, S_1 leaves and S_2 enters. Pivot element is $3/2$ in S_1 row.

3. Pivot (S_1 row, S_2 column, element = $3/2$):

- Divide S_1 row by $3/2$ to make pivot = 1:
New S_1 : $[0, 0, 0, 2/3, 1, -1/3, 16.666]$
So, S_2 row becomes (after swap):
 $| S_2 | 0 | 0 | 0 | 2/3 | 1 | -1/3 | 16.666\dots |$
- Update all other rows to make S_2 column = 0 except S_2 row.

Perform row operations on Z, x_2 , x_1 .

Mathematics for Computer Science Engineers

Simplex Method Example 1



Z-row: Current [1, 0, 0, 0, -15, 45, 7350]

Z-row update: $Z_{\text{new}} = Z_{\text{old}} + 15 \times (\text{new } S_2 \text{ row})$

- S_2 coeff in Z = -15, so add $15 \times S_2$ row to Z

Each entry:

- $S_1: 0 + 15 \times (2/3) = 10$
- $S_2: -15 + 15 \times 1 = 0$
- $S_3: 45 + 15 \times (-1/3) = 45 - 5 = 40$
- RHS: $7350 + 15 \times 16.6667 = 7600$

So Z row becomes:

| Z | 1 | 0 | 0 | 10 | 0 | 40 | 7600 |

x_2 -row: Current

x_2 row update: $x_{2\text{-new}} = x_{2\text{-old}} - 1 \times (\text{new } S_2 \text{ row})$ (since S_2 coefficient in x_2 row = 1)

Each entry:

- $S_1: 0 - (2/3) = -2/3$
- $S_2: 1 - 1 = 0$
- $S_3: 0 - (-1/3) = 1/3$
- RHS: $50 - 16.6667 = 33.3333$

So x_2 row:

| x_2 | 0 | 0 | 1 | -2/3 | 0 | 1/3 | 33.3333 |

Mathematics for Computer Science Engineers

Simplex Method Example 1



x_1 -row: Current $[0, 1, 0, 0, -3/2, 1/2, 15]$

x_1 row update: $x_{1_new} = x_{1_old} + (3/2) \times (\text{new } S_2 \text{ row})$

- S_2 coefficient in x_1 row = $-3/2$, so: $x_{1_new} = x_{1_old} + (3/2) \times (\text{new } S_2 \text{ row})$:

But this is negative, double-check with sign:

- x_{1_old} : $[0, 1, 0, 0, -3/2, 1/2, 15]$
- S_2 coeff = $-3/2$, so $x_{1_new} = x_{1_old} + (3/2) \times (\text{new } S_2 \text{ row})$
- Or, $x_{1_new} = x_{1_old} - (3/2) \times (\text{new } S_2 \text{ row})$

So, for each:

- S_1 : $0 - (-3/2) \times (2/3) = 0 + 1 = 1$
- S_2 : $-3/2 - (-3/2) \times 1 = -3/2 + 3/2 = 0$
- S_3 : $1/2 - (-3/2) \times (-1/3) = 1/2 - 1/2 = 0$
- RHS: $15 - (-3/2) \times 16.6667 = 15 + 25 = 40$

So x_1 row:

$| x_1 | 0 | 1 | 0 | 1 | 0 | 0 | 40 |$

Mathematics for Computer Science Engineers

Simplex Method Example 1



Final solution (read off the updated table)

- Basic variable values match the RHS of their rows.
- $x_1 = 40$
- $x_2 = 33.33$
- $s_1 = 0$
- $s_2 = 16.67$

Objective value: $Z_{\max} = 7600$

Check: $Z = 90(40) + 120(33.33) = 3600 + 3999.6 = 7600$.

Check constraints:

- $x_1 = 40 \leq 40$
- $x_2 = 33.33 \leq 50$
- $2(40) + 3(33.33) = 80 + 99.99 = 180$
- Remaining land by constraint 2: $s_2 = 16.67$, (so x_2 does not fully exhaust land limit).

Optimal planting: Plant **40 hectares of pine** and **33.33 hectares of eucalypt**.

Maximum profit: \$7,600.

Basis	x_1	x_2	s_1	s_2	s_3	RHS
Z	0	0	10	0	40	7600
s_2	0	0	$2/3$	1	$-1/3$	16.67
x_2	0	1	$-2/3$	0	$1/3$	33.33
x_1	1	0	1	0	0	40

Mathematics for Computer Science Engineers

Simplex Method Example 2



A furniture company manufactures **three types of products**:

- x_1 : Tables, x_2 : Chairs, x_3 : Cabinets

Each product contributes a different profit per unit:

- Each **table** earns a profit of **\$70**,
Each **chair** earns **\$80**,
Each **cabinet** earns **\$60**.

However, production is limited by **available materials and resources**, as follows:

1. Due to a shortage of **high-quality wood**, the factory can produce **at most 30 tables**.
2. Because of the limited number of **metal frames**, **chair production cannot exceed 40 units**.
3. The total **workshop capacity (labour hours, machine time, and finishing space)** restricts overall production — producing
 - 1 table uses **1 capacity unit**,
 - 1 chair uses **2 capacity units**, and
 - 1 cabinet uses **1 capacity unit**, and the **total capacity available is 100 units**.

The company wants to determine how many **tables (x_1)**, **chairs (x_2)**, and **cabinets (x_3)** to produce in order to **maximize total profit**.

Simplex Method Example 2

Maximize $Z = 70x_1 + 80x_2 + 60x_3$, where $x_1, x_2, x_3 \geq 0$, subject to constraints:

$$x_1 \leq 30$$

$$x_2 \leq 40$$

$$x_1 + 2x_2 + x_3 \leq 100$$

Introduce slack variables S_1, S_2, S_3 (all ≥ 0) to convert to equations:

$$x_1 + S_1 = 30$$

$$x_2 + S_2 = 40$$

$$x_1 + 2x_2 + x_3 + S_3 = 100$$

Objective as equation (canonical):

$$Z - 70x_1 - 80x_2 - 60x_3 = 0.$$

Mathematics for Computer Science Engineers

Simplex Method Example 2

Initial simplex tableau (columns: [Z, x_1 , x_2 , x_3 , S_1 , S_2 , S_3 | RHS])

Basic Var.	Z	x_1	x_2	x_3	S_1	S_2	S_3	RHS
Z	1	-70	-80	-60	0	0	0	0
S_1	0	1	0	0	1	0	0	30
S_2	0	0	1	0	0	1	0	40
S_3	0	1	2	1	0	0	1	100

Basic variables initially: $S_1 = 30$, $S_2 = 40$, $S_3 = 100$.

Z-row has negative coefficients under x_1 , x_2 , x_3 , so improvement is possible.

Mathematics for Computer Science Engineers

Simplex Method Example 2

Step 1 — First pivot (enter x_2)

Pick entering variable = the most negative in Z-row: -80 under $x_2 \Rightarrow x_2$ enters.

Ratio test (only rows with positive coefficient in x_2):

- S_1 row: coeff 0 $\rightarrow$ not eligible
- S_2 row: coeff 1 $\rightarrow$ ratio = RHS / coeff = $40 / 1 = 40$
- S_3 row: coeff 2 $\rightarrow$ ratio = $100 / 2 = 50$

Minimum ratio = 40 $\Rightarrow$ **S_2 leaves**, pivot element = 1 at (S_2, x_2) .

Pivot row (S_2) already has 1 in x_2 so no scaling needed.

Eliminate x_2 from all other rows:

Eliminate x_2 from all other rows:

- New Z-row = old Z + $80 \times (S_2 \text{ row})$
 - old Z = $[1, -70, -80, -60, 0, 0, 0 \mid 0]$
 - $80 \times S_2 = [0, 0, 80, 0, 0, 80, 0 \mid 3200]$
 $\rightarrow$ new Z = $[1, -70, 0, -60, 0, 80, 0 \mid 3200]$
- S_1 unchanged (x_2 coeff = 0).
- New S_3 = old $S_3 - 2 \times (S_2 \text{ row})$
 - old $S_3 = [0, 1, 2, 1, 0, 0, 1 \mid 100]$
 - $2 \times S_2 = [0, 0, 2, 0, 0, 2, 0 \mid 80]$
 $\rightarrow$ new $S_3 = [0, 1, 0, 1, 0, -2, 1 \mid 20]$

Rename S_2 row $\rightarrow x_2$ row (basic variable now $x_2 = 40$).

Tableau after Step 1:

Basic Var.	Z	x_1	x_2	x_3	s_1	s_2	s_3	RHS
Z	1	-70	0	-60	0	80	0	3200
s_1	0	1	0	0	1	0	0	30
x_2	0	0	1	0	0	1	0	40
s_3	0	1	0	1	0	-2	1	20

Interpretation: x_2 has entered the basis with value $x_2 = 40$. Z improved from 0 to 3200 (that's the value in Z-row RHS after the row operation).

Step 2 — Second pivot (enter x_1)

Z-row reduced costs for decision variables: $x_1: -70$, $x_2: 0$, $x_3: -60$. Most negative is $-70 \Rightarrow x_1$ **enters**.

Ratio test for x_1 (rows with positive x_1 coefficient):

- S_1 row: coeff 1 $\rightarrow$ ratio = $30 / 1 = 30$
- x_2 row: coeff 0 $\rightarrow$ not eligible
- S_3 row: coeff 1 $\rightarrow$ ratio = $20 / 1 = 20$

Minimum ratio = 20 $\Rightarrow$ **S_3 leaves**, pivot element = 1 at (S_3, x_1) .

Pivot row S_3 already has 1 in x_1 — it will become the new x_1 row.

Eliminate x_1 from other rows:

- New $Z = \text{old } Z + 70 \times (\text{new } x_1 \text{ row})$
 - old $Z = [1, -70, 0, -60, 0, 80, 0 \mid 3200]$
 - $70 \times S_3 = 70 \times [0, 1, 0, 1, 0, -2, 1 \mid 20] = [0, 70, 0, 70, 0, -140, 70 \mid 1400]$
 $\rightarrow \text{new } Z = [1, 0, 0, 10, 0, -60, 70 \mid 4600]$
- New $S_1 = \text{old } S_1 - 1 \times S_3$
 - old $S_1 = [0, 1, 0, 0, 1, 0, 0 \mid 30]$
 - $S_3 = [0, 1, 0, 1, 0, -2, 1 \mid 20]$
 $\rightarrow \text{new } S_1 = [0, 0, 0, -1, 1, 2, -1 \mid 10]$
- x_2 row unchanged.

Rename $S_3 \rightarrow x_1$ (basic variable now $x_1 = 20$).

Mathematics for Computer Science Engineers

Simplex Method Example 2

Tableau after Step 2 (final tableau):

Basic Var.	Z	x_1	x_2	x_3	s_1	s_2	s_3	RHS
Z	1	0	0	10	0	-60	70	4600
s_1	0	0	0	-1	1	2	-1	10
x_2	0	0	1	0	0	1	0	40
x_1	0	1	0	1	0	-2	1	20

Step 3 — Third pivot (enter S2)

Entering variable: S2 (most negative coefficient in Z-row: -60)

Ratio test (rows with positive S2 coefficient):

- S1 row: coeff = 2 $\rightarrow$ ratio = $10/2 = 5$
- x2 row: coeff = 1 $\rightarrow$ ratio = $40/1 = 40$
- x1 row: coeff = -2 $\rightarrow$ negative, not eligible

Minimum ratio = 5 $\Rightarrow$ S1 leaves, pivot element = 2 at (S1, S2)

Scale pivot row: Divide S1 row by 2:

- New row: $[0, 0, -1/2, 1/2, 1, -1/2 \mid 5]$

Mathematics for Computer Science Engineers

Simplex Method Example 2

Eliminate S2 from other rows:

New Z-row = old Z + 60 × (new S2 row)

- old Z = [1, 0, 0, 10, 0, -60, 70 | 4600]
- 60 × new S2 = [0, 0, 0, -30, 30, 60, -30 | 300]
- **New Z = [1, 0, 0, -20, 30, 0, 40 | 4900]**

New x2 row = old x2 - 1 × (new S2 row)

- old x2 = [0, 0, 1, 0, 0, 1, 0 | 40]
- new S2 = [0, 0, 0, -1/2, 1/2, 1, -1/2 | 5]
- **New x2 = [0, 0, 1, 1/2, -1/2, 0, 1/2 | 35]**

New x1 row = old x1 + 2 × (new S2 row)

- old x1 = [0, 1, 0, 1, 0, -2, 1 | 20]
- 2 × new S2 = [0, 0, 0, -1, 1, 2, -1 | 10]
- **New x1 = [0, 1, 0, 0, 1, 0, 0 | 30]**

Mathematics for Computer Science Engineers

Simplex Method Example 2

Tableau after Step 3:

Basic Var.	Z	x1	x2	x3	s1	s2	s3	RHS
Z	1	0	0	-20	30	0	40	4900
s2	0	0	0	-1/2	1/2	1	-1/2	5
x2	0	0	1	1/2	-1/2	0	1/2	35
x1	0	1	0	0	1	0	0	30

Still not optimal! The Z-row has **-20 under x3**, so we need another iteration.

Mathematics for Computer Science Engineers

Simplex Method Example 2

Step 4 — Fourth pivot (enter x_3)

Entering variable: x_3 (coefficient -20 in Z-row)

Ratio test:

- S2 row: coeff = $-1/2 \rightarrow$ negative, not eligible
- x_2 row: coeff = $1/2 \rightarrow$ ratio = $35 / (1/2) = 70$
- x_1 row: coeff = $0 \rightarrow$ not eligible

x_2 leaves, pivot element = $1/2$ at (x_2 , x_3)

Scale pivot row: Multiply x_2 row by 2:

- New x_3 row: $[0, 0, 2, 1, -1, 0, 1 \mid 70]$

Eliminate x_3 :

New $Z = \text{old } Z + 20 \times (\text{new } x_3 \text{ row})$

- **New $Z = [1, 0, 40, 0, 10, 0, 60 \mid 6300]$**

New $S2 = \text{old } S2 + (1/2) \times (\text{new } x_3 \text{ row})$

- **New $S2 = [0, 0, 1, 0, 0, 1, 0 \mid 40]$**

x_1 unchanged (x_3 coeff = 0)

Final Optimal Tableau:

Basic Var.	Z	x1	x2	x3	S1	S2	S3	RHS
Z	1	0	40	0	10	0	60	6300
S2	0	0	1	0	0	1	0	40
x3	0	0	2	1	-1	0	1	70
x1	0	1	0	0	1	0	0	30

All Z-row coefficients for decision variables are non-negative!

Optimal Solution:

- **$x1 = 30$**
- **$x2 = 0$ (nonbasic)**
- **$x3 = 70$**
- **Maximum $Z = 6300$**

1. Tie for the Entering Basic Variable (BV)

- In a **maximization problem**, the entering variable is the **non-basic variable (NBV)** with the most negative coefficient in **Z row (objective function row)**.
- Sometimes, in the very initial table or after a few iterations, **two or more NBVs have the same most negative coefficient** → tie.
- In this case, **we can choose any of them arbitrarily** as the entering variable.

Example:

Maximize: $Z = 5x_1 + 5x_2$

Subject to:

- $x_1 + x_2 \leq 4$
- $2x_1 + x_2 \leq 5$
- $x_1, x_2 \geq 0$

BV	x_1	x_2	s_1	s_2	RHS
Z	-5	-5	0	0	0
x_1	1	1	1	0	4
x_2	2	1	0	1	5

Look at **Row Z**: Both x_1 and x_2 have same coefficient → **tie**.

We can choose **either x_1 or x_2** arbitrarily as the entering variable.

Odd cases in Simplex method

2. Tie for the Leaving Basic Variable (Degenerate Case)

Theory:

- After choosing the entering variable, we perform **minimum ratio test** to select the leaving BV:

$$\text{Ratio} = \frac{b_i}{a_{ij}}$$

- Tie occurs:** Two rows have the same minimum positive ratio.
- Degenerate solution:** A basic variable becomes zero after pivot $\rightarrow$ called **degenerate**.
- In tie cases, **select arbitrarily**.

Suppose we have the table:

Entering variable: x_1 .

Minimum ratio test:

Row 2: $8 / 2 = 4$

Row 3: $4 / 1 = 4$

BV	x_1	x_2	s_1	s_2	RHS
Z	-3	-2	0	0	0
s_1	2	2	0	1	8
s_2	1	-3	0	0	4

- Tie for leaving variable $\rightarrow$ choose **any**, says s_1 leaves.

Degeneracy: If after pivot, RHS = 0 for a Basic Variable, that Basic variable is **degenerate**.

Degenerate solutions **do not increase Z**, but Simplex continues.

Odd cases in Simplex method

3. No Leaving BV $\rightarrow$ Unbounded Z

Theory:

- Occurs when **all entries in pivot column ≤ 0** (ignoring Row 0).
- Meaning: no constraint limits increase of entering variable $\rightarrow$ Z can increase indefinitely $\rightarrow$ **unbounded solution**.

Example:

Maximize: $Z = x_1 + x_2$

Subject to:

- $x_1 - x_2 \leq 2$
- $-x_1 + 2x_2 \leq -1$
- $x_1, x_2 \geq 0$

If we try x_2 as entering variable:

BV	x_1	x_2	s_1	s_2	RHS
Z	-1	-1	0	0	0
s_1	1	-1	1	0	2
s_2	-1	2	0	1	-1

- Pivot column for x_2 : entries = -1, 2
- If all non basic variables $\leq 0 \rightarrow$ **solution is unbounded**.

4. Multiple Optimal Solutions

Theory:

- Occurs when **any NBV in Z row has zero coefficient** $\rightarrow$ adding that variable does not change Z.
- Means there are **alternate optimal solutions**.
- Perform **one more iteration** to find the other optimal solution.

Example:

BV	x_1	x_2	s_1	s_2	RHS
Z	0	0	1	1	10
s_1	1	0	0.5	-0.5	2
s_2	0	1	0.5	0.5	2

- Z row: Coefficient of non-basic variables = 0 $\rightarrow$ **alternate optimal solutions exist**.
- Any combination of NBVs keeping $Z = 10 \rightarrow$ still optimal.

Mathematics for Computer Science Engineers

Simplex Method Steps for Minimization Type Problems



1. Formulate the LP problem: Write objective function $Z = c_1x_1 + c_2x_2$ and constraints.
2. Convert constraints to equalities:
 - For $\geq$ constraints, subtract surplus variables and add artificial variables.
3. Convert to maximization (optional):
 - Maximize $-Z$ if you prefer standard Simplex.
4. Set up initial Simplex tableau:
 - Include decision, surplus, and artificial variables.
 - Use Big M method or Two-Phase method for artificial variables.
5. Select entering variable:
 - Most negative coefficient in row Z (objective function row).
6. Select leaving variable:
 - Apply minimum ratio test (RHS/pivot column) for positive coefficients.
7. Pivot:
 - Make pivot = 1 and other entries in pivot column = 0 using row operations.
8. Repeat iterations:
 - Continue until no negative coefficients in row Z $\rightarrow$ optimal solution reached.

Mathematical formulation

Minimize

$$Z = 50x_1 + 40x_2 + 60x_3$$

subject to

$$x_1 + x_2 + x_3 \geq 60 \quad (1)$$

$$2x_1 + x_2 + 3x_3 \geq 90 \quad (2)$$

$$x_1 + 2x_2 + x_3 \leq 100 \quad (3)$$

$$x_1, x_2, x_3 \geq 0.$$

Convert inequalities to equalities using surplus, slack, and artificial variables:

$$x_1 + x_2 + x_3 - s_1 + a_1 = 60 \quad (a_1 \text{ artificial})$$

$$2x_1 + x_2 + 3x_3 - s_2 + a_2 = 90 \quad (a_2 \text{ artificial})$$

$$x_1 + 2x_2 + x_3 + s_3 = 100.$$

If problem is of minimization type and the constraints are $\geq$, then we can solve it using the **Big M method**.

NOTE: Big M method is out of the scope of our syllabus

1.

A company produces 3 different products: A, B and C. Each product has to go under 3 processes consuming different amounts of time along the way. The time available for each process is described in the table below.

Process	Total Hours Available	Hours per Unit Product A	Hours per Unit Product B	Hours per Unit Product C
I	12,000	5	2	4
II	24,000	4	5	6
III	18,000	3	5	4

Assuming the selling profits for products A, B and C are 2, 3 and 4€ per unit. Determine how many units of each product should be produced to maximize the profit. Was there any time left?

2.

A company produces 3 different bookshelves: a luxury, a regular and an exportation model. Consider the maximum demand for each model to be 500, 750 and 400 respectively. The working hours at the carpentry and finishing sections have the working time limitations

Assuming the selling profit for the luxury, regular and exportation models is 1500, 1300 2500 respectively, formulate the LP problem in order to maximize the profit. Interpret the results detailing the optimal number of bookshelves of each type produced discussing the total amount of hours used in each section. How far from meeting the maximum demands were we?

Section	Total Hours Available (thousands)	Hours per Unit Luxury	Hours per Unit Regular	Hours per Unit Exportation
Carpentry	1.4	0.5	0.5	1.0
Finishing	1.2	0.5	0.5	2.0

3.

$$\text{Max: } Z = x_1 + 2x_2$$

$$\text{Subject to: } 2x_1 + 4x_2 \leq 20$$

$$x_1 + x_2 \leq 8 \text{ and } x_1, x_2 \geq 0$$

4.

$$\text{Max: } Z = x_1 + x_2$$

$$\text{Subject to: } x_1 + x_2 \leq 4$$

$$2x_1 + x_2 \leq 6$$

$$x_1 + 2x_2 \leq 6 \text{ and } x_1, x_2 \geq 0$$

5.

$$\text{Max: } Z = x_1 + x_2$$

$$\text{Subject to: } x_1 + x_2 \leq 10$$

$$2x_1 - 3x_2 \leq 15$$

$$x_1 - 2x_2 \leq 20 \text{ and } x_1, x_2 \geq 0$$

6.

$$\text{Max: } Z = -x_2$$

$$\text{Subject to: } x_1 + x_2 + x_3 \leq 100$$

$$x_1 - 5x_2 \leq 40$$

$$x_3 \geq -10 \text{ and } x_1, x_2, x_3 \geq 0$$

7.

$$\text{Min: } Z = 2x_1 - 3x_2 - 4x_3$$

$$\text{Subject to: } x_1 + 5x_2 - 3x_3 \leq 15$$

$$x_1 + x_2 + x_3 \leq 11$$

$$5x_1 - 6x_2 + x_3 \leq 4 \text{ and } x_1, x_2, x_3 \geq 0$$

8.

$$\text{Max: } Z = 10x_1 + 30x_2$$

$$\text{Subject to: } x_1 \leq 15$$

$$x_1 - x_2 \leq 20$$

$$-3x_1 + x_2 \leq -30 \text{ and } x_1, x_2 \geq 0$$

1. Engineering Optimization Theory and Practice, Fifth Edition, Singiresu S. Rao, 2020.



THANK YOU

Chandravva Hebbi

Computer Science and Engineering



PES
UNIVERSITY

CELEBRATING 50 YEARS

Mathematics for Computer Science Engineers

Unit - 4 Discrete variable optimization

Chandravva Hebbi

Computer Science and Engineering

Discrete variable optimization

Discrete Variable Optimization involves finding the best solution where decision variables can only take discrete values. These values are often integers or specific finite choices.

Types of Discrete Variable Optimization

1. Combinatorial Optimization

- Focuses on finding the best combination of discrete objects chosen from a finite set. Eg. Traveling Salesman Problem, Knapsack Problem

2. Integer Programming (IP)

- Optimization where some or all variables are restricted to integer values.
- Important in problems where quantities cannot be fractional like number of machines, workers, or batches.

Note:

- Similar to Linear Programming models but with integer restrictions on decision variables.
- The restrictions ensure solutions are realistic (e.g., you cannot purchase a fraction of a machine).

Types of Integer Programming Models:

- **Total Integer Model:** All decision variables are integers. (Similar to Linear Programming models but with integer restrictions on decision variables)
- **0-1 Integer Model:** All decision variables are binary; values are either 0 or 1 (commonly used for yes/no decisions).
- **Mixed Integer Model:** Some variables are integers, others can be continuous/real numbers.

Mathematics for Computer Science Engineers

Total Integer Problem Formulation



Press machine increases profit by \$100, Lathe machine increases profit by \$150

Price and floor space:

- Press: \$8,000 and 15 sq ft space
- Lathe: \$4,000 and 30 sq ft space

Budget: \$40,000

Floor space: 200 sq ft

Goal: Maximize profit by deciding how many presses and lathes to purchase.

Problem Formulation:

x_1 = number of press machines (integer, ≥ 0) , x_2 = number of lathe machines (integer, ≥ 0)

Objective Function:

Maximize $Z = 100x_1 + 150x_2$

Constraints:

$8000x_1 + 4000x_2 \leq 40000$ (Budget constraint) , $15x_1 + 30x_2 \leq 200$ (Floor space constraint)

Variable restrictions:

$x_1, x_2 \geq 0$, both integers $x_1, x_2 \geq 0$, both integers

0-1 Integer Model Example

Problem Context:

- Research and Development (R&D) program with 4 projects: A, B, C, and D.
- Constraints on the number of Operations Research (OR) specialists and budget.
- Expected profit, cost, and specialist requirements vary per project.
- Special condition: Only project A or project B can be chosen, not both.

Key Elements:

Decision Variables:

- Binary variables $x_i \in \{0,1\}$ for each project i , where 1 means selecting the project, 0 means not selecting.

Constraints:

- Budget constraints based on project costs.
- Specialist availability constraints.
- Mutually exclusive project selection for projects A and B.

Project	Expected cost	OR Specialists required	Expected Profit (thousands of dollars)
A	35,000	4	300
B	10,000	2	90
C	25,000	7	400
D	90,000	3	150

0-1 Integer Model Example

Problem Setup:

- Four research projects: A, B, C, and D
- Each project has expected profit, cost, and resource requirements
- Projects can either be selected (1) or not selected (0)
- **Special constraint:** Projects A and B are mutually exclusive (only one can be chosen)

Model Formulation:

Decision Variables:

- $A, B, C, D \in \{0, 1\}$ where:
 - $A = 1$ if project A is selected, 0 otherwise
 - $B = 1$ if project B is selected, 0 otherwise
 - $C = 1$ if project C is selected, 0 otherwise
 - $D = 1$ if project D is selected, 0 otherwise

0-1 Integer Model Example

Objective Function:

Maximize $Z = 300A + 90B + 400C + 150D$

Constraints:

1. Budget constraint:

$$35,000A + 10,000B + 25,000C + 90,000D \leq 120,000$$

2. OR Specialists constraint:

$$4A + 2B + 7C + 3D \leq 12$$

Mutually exclusive projects:

$$A + B = 1$$

This ensures either A or B is selected, but not both.

3. Binary restrictions:

$$A, B, C, D \in \{0,1\}$$

Key Insight:

The constraint $A + B = 1$ means exactly one of projects A or B must be chosen. This would result in solutions like $A = 1, B = 0$ or $A = 0, B = 1$, but never both equal to 1.

Mathematics for Computer Science Engineers

Mixed Integer Model



Problem Description:

- **Budget:** \$500,000 to invest in townhouses and land
- **Returns:** \$8,000 per townhouse, \$6,000 per acre of land
- **Costs:** Townhouse = \$70,000, Land = \$30,000 per acre
- **Maintenance:** \$1,000 per townhouse, \$2,000 per acre annually
- **Maintenance budget:** \$14,000
- **Goal:** Maximize annual return

Decision Variables:

- x_1 = number of townhouses (must be integer)
- x_2 = acres of land (can be fractional)

Model Formulation:

Objective Function:

$$\text{Maximize } Z = 8000x_1 + 6000x_2$$

Constraints:

1. Investment budget:

$$70,000x_1 + 30,000x_2 \leq 500,000$$

1. Maintenance budget:

$$1000x_1 + 2000x_2 \leq 14,000$$

Variable Restrictions:

- $x_1 \geq 0$ and integer (townhouses must be whole units)
- $x_2 \geq 0$ (land can be fractional acres)

Key Features:

- **Mixed nature:** One variable must be integer, the other can be continuous
- **Real-world logic:** You can't build half a townhouse, but you can buy fractional acres

Mathematics for Computer Science Engineers

Integer Programming Solution Issues



The Rounding Problem:

Example from Machine Purchase Problem:

- LP solution: $x_1 = 2.22, x_2 = 5.55$
- Rounded solution: $x_1 = 2, x_2 = 6$

Problem: When substituting rounded values back into constraints:

- Original constraint: $15x_1 + 30x_2 \leq 200$
- Check: $15(2) + 30(6) = 30 + 180 = 210 > 200$
- **Result:** Constraint is violated → **Infeasible solution**

Why Rounding Fails:

- Rounding may violate budget, space, or resource constraints
- The optimal integer solution may be far from the LP solution
- Need specialized integer programming algorithms like **Branch-and-Bound**

Mathematics for Computer Science Engineers

Branch-and-Bound Method for Integer Programming

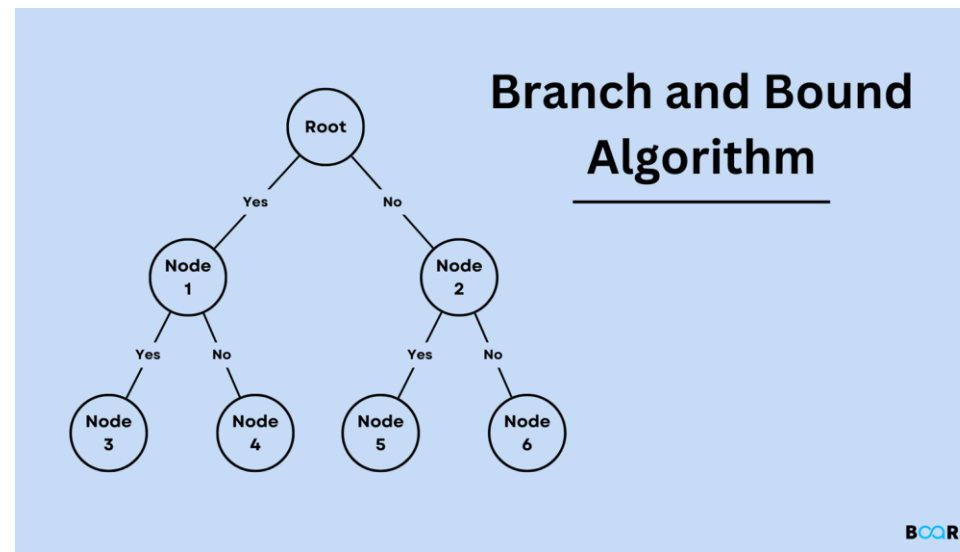
Concept Overview:

Branch-and-bound is a systematic algorithm to solve integer programming problems by dividing the problem into smaller subproblems.

Branch: Refers to constructing a tree structure where each node represents a subproblem with some variables restricted.

Bound: Uses an estimate of the objective function (a bound) to prune subproblems that cannot yield better solutions than the current best.

This approach avoids exhaustive search by ignoring infeasible or suboptimal branches early.



Mathematics for Computer Science Engineers

Branch-and-Bound Method for Integer Programming



When to Stop Branching or Prune?

1. When an **integer feasible solution** is found.
2. When a node is **infeasible**.
3. When further branching **cannot improve objective z**:
 - For minimization, no potential for a lower z
 - For maximization, no potential for a higher z

Note: Branch and Bound Method is applicable for any number of variables.

Branch-and-Bound Process Steps:

1. Start with the LP relaxation of the integer problem (ignore integrality).
2. If the LP solution is integer, it is optimal—stop.
3. Otherwise, select a variable with a fractional value and create two branches:
 - One branch with the variable rounded down.
 - Another with the variable rounded up.
4. Solve subproblems recursively, calculating bounds.
5. Prune branches where bounds show no improvement over the best-found solution.
6. Continue until all branches are either solved or pruned.

Mathematics for Computer Science Engineers

Branch-and-Bound Examples

Example:

Decision variables: $x = [x_1, x_2]$

- $x_1 \in \{0, 1, 2, 3, 4, 5, 6, 7, 8\}$
- $x_2 \in \{0, 1, 2, 3, 4, 5\}$

Maximize, $z = 3x_1 + 5x_2$

Constraints:

$$2x_1 + 4x_2 \leq 25$$

$$x_1 \leq 8 \text{ and } 2x_2 \leq 10$$

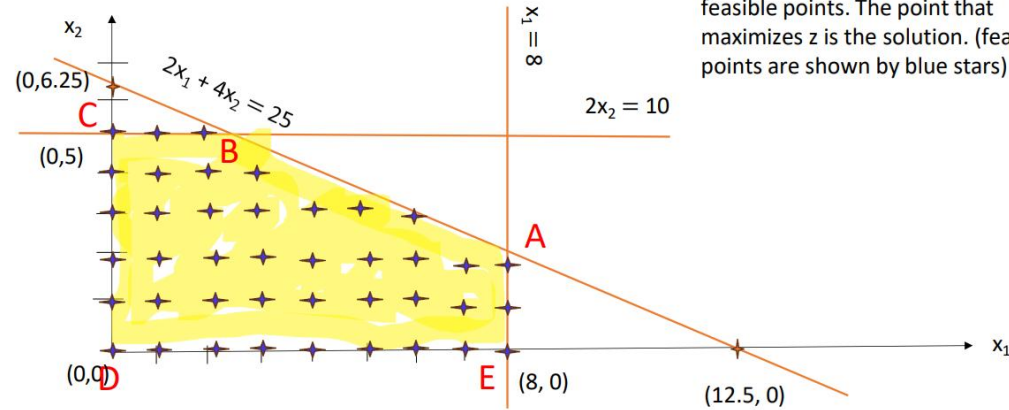
Determine: x , c , A , b

Step 1:

Relax the Integer Constraints

- Initially, ignore the integer restrictions on x_1 and x_2 .
- Solve the relaxed linear optimization problem allowing x_1, x_2 to be any real (continuous) values.

Solution: $x_1 = 8, x_2 = 2.25$, with objective value $z = 35.25$.



All the (x_1, x_2) points which fall in the yellow region and are integers are the feasible points. The point that maximizes z is the solution. (feasible points are shown by blue stars)

$$\begin{aligned} x_1 &= 8 \\ x_2 &= 2.25 \\ z &= 35.25 \end{aligned}$$

Level 1

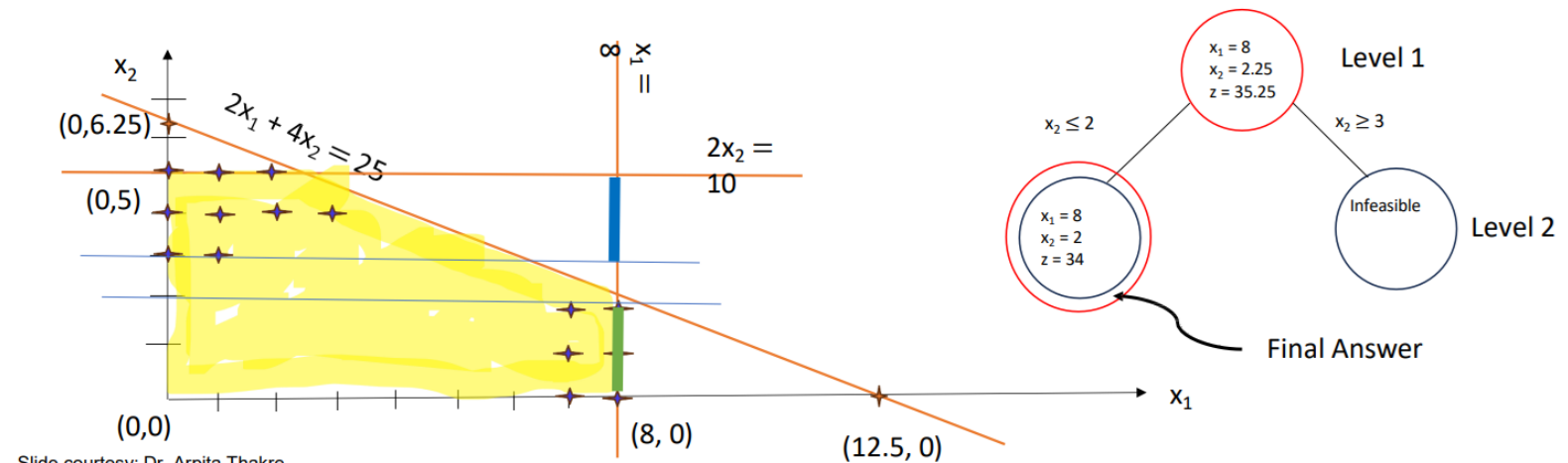
Step 2: Identify Solution vs. Original Problem

- The solution maximizes z but is **not valid for the original integer problem** because $x_2 = 2.25$ is fractional.
- To understand this, evaluate z at corner points (such as A, B, C, D, E).
- Point A yields the highest z , but review if x_1, x_2 satisfy integer constraints (they do not fully).

Step 3: Branching From Relaxed Solution

- **Branch by enforcing integer values on one variable at a time**, keeping the other continuous.
- Partition the feasible region (yellow area) into two **disjoint subregions**:

1. $x_1 = 8$ and $x_2 \leq 2$ (green line region)
2. $x_1 = 8$ and $x_2 \geq 3$ (blue line region)



Step 4: Solve Subproblems at Next Level

At **level 2** of branching, check subproblems:

One node is **infeasible** (no solution satisfies constraints).

The other node yields an integer solution: $x_1 = 8$, $x_2 = 2$, $z = 34$.

Both x_1 and x_2 are integers and satisfy all constraints.

Therefore, **stop branching** as integer feasibility and constraints are met.

Key Observations

- This example was solved quickly with **just one level of branching**.
- Because of the immediate integer feasible solution, **no pruning occurred** in this case.
- The value $z=35.25$ from the relaxed problem is an **upper bound** on the maximum integer solution achievable.

Mathematics for Computer Science Engineers

Branch and Bound Example

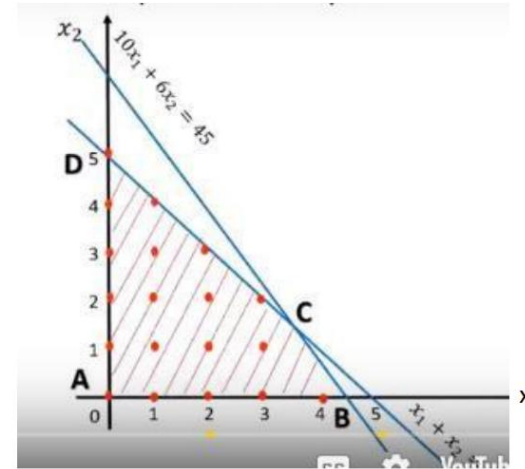
Example 2

Problem Setup

Maximize: $z = 5x_1 + 4x_2$

Subject to: $x_1 + x_2 \leq 5$ and $10x_1 + 6x_2 \leq 45$

Decision variables x_1, x_2 can only take **positive integer values**.



All the (x_1, x_2) points which fall in the shaded region and are integers are the feasible points. The point that maximizes z is the solution. (feasible points are shown by red dots)

Step 1: Relax Integer Constraints and Solve

Relax the integer restriction on x_1, x_2 allowing them to be continuous.

Solve the relaxed linear programming problem.

Solution: $x_1 = 3.75, x_2 = 1.25, z = 23.75$.

This solution is found by evaluating z at the corner points of the feasible region formed by the constraints.

We notice **point C** gives the highest z value among all corners.

However, this solution is **not valid for the original integer problem** because x_1 and x_2 are not integers.

The value $z=23.75$ is an **upper bound** for the integer problem's optimal value.

Mathematics for Computer Science Engineers

Branch and Bound Example

Step 2: Branching

Branch by fixing one variable to integer values and allowing the other to remain continuous.

Partition the feasible region (shaded area) into two **disjoint subregions** based on x_2 :

- $x_1 \leq 3$ (green area)

- $x_1 \geq 4$ (blue area)

$$\begin{aligned} x_1 &= 3.75 \\ x_2 &= 1.25 \\ z &= 23.75 \end{aligned}$$

Level 1

Step 3: Analyse Level 2 Nodes

When $x_1 \leq 3$ (fix $x_1 = 3$):

- Best integer solution at this node:

$$x_1 = 3, x_2 = 2, z = 23.$$

- Both x_1 and x_2 are integers and satisfy constraints.

- No further branching necessary** for this node.

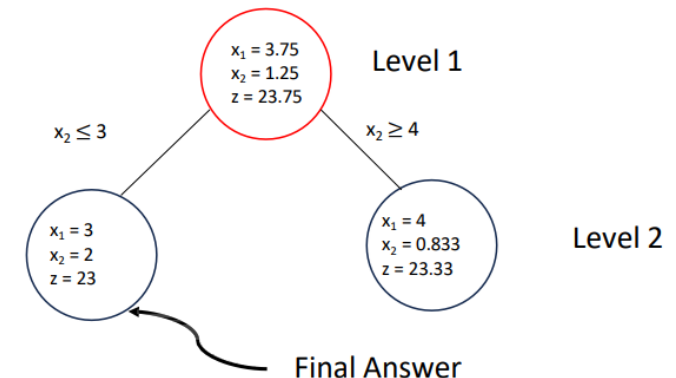
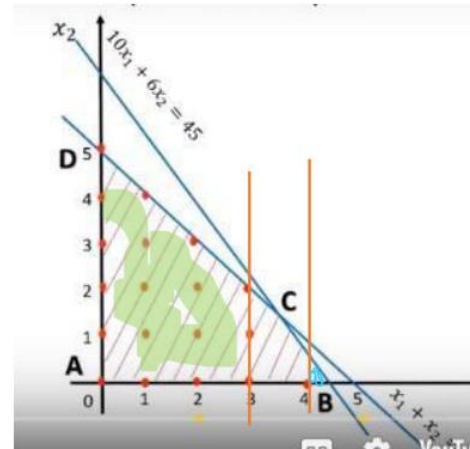
When $x_2 \geq 4$ (fix $x_1 = 4$):

- Best solution (not integer for x_2):

$$z = 23.33 \text{ at } x_2 = 0.833 \text{ (non-integer).}$$

- Should we branch further here?

- No**, because branching will only decrease z .



Future branching cannot yield $z \geq 24$.

Mathematics for Computer Science Engineers

Branch and Bound Example

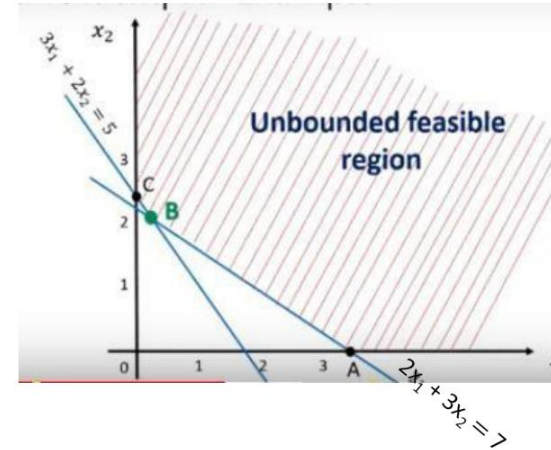
Example 3

Problem Setup

Minimize: $z = 5x_1 + 4x_2$

Subject to: $3x_1 + 2x_2 \geq 5$ and $2x_1 + 3x_2 \geq 7$

Decision variables x_1, x_2 must be **positive integers**.



All the (x_1, x_2) points which fall in the shaded region and are integers are the feasible points. The point that maximizes z is the solution.

Step 1: Relax Integer Constraint and Solve

Relax integer restrictions, allowing x_1, x_2 continuous values.

Solve the relaxed problem.

Solution: $x_1 = 0.2, x_2 = 2.2$ objective $z = 9.8$.

Evaluate z at corner points (A, B, C).

Point B gives the **lowest value of z** .

However, this solution is **not valid for original integer problem**.

$z=9.8$ serves as a **lower bound** on the true minimum.

$$\begin{aligned} x_1 &= 0.2 \\ x_2 &= 2.2 \\ z &= 9.8 \end{aligned}$$

Level 1

Mathematics for Computer Science Engineers

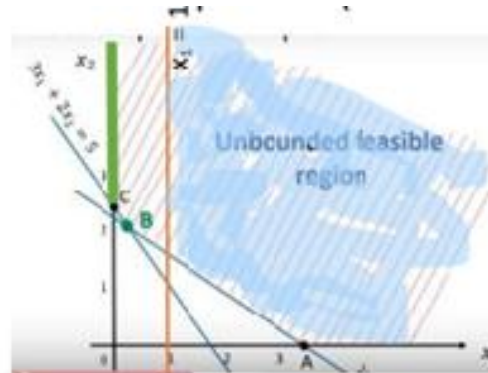
Branch and Bound Example

Step 2: Branch Preparation

Branch by fixing one variable to an integer, leaving the other continuous.

The feasible region (shaded) is partitioned into **two disjoint parts**:

- An unbounded region
- A constraint line dividing these regions.

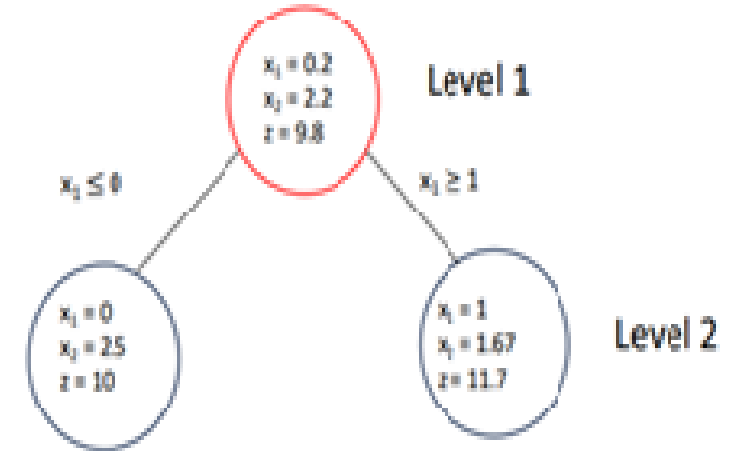


Step 3: Node Analysis at Level 2

Two nodes:

- **Left node:** $x_1 = 0, x_2 = 2.5, z=10$.
- **Right node:** $x_1 = 1, x_2 = 1.67, z=11.7$.

Need to decide which node to branch next.



Step 4: Branching Decision Rule

For **minimization problems**, branch on the node with the **smallest z value** first (best bound).

This improves efficiency by focusing search on promising regions.

Mathematics for Computer Science Engineers

Branch and Bound Example

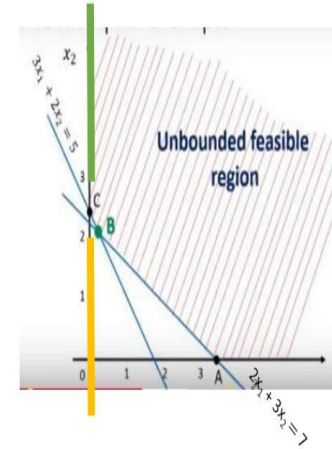
Step 5: Branching at Level 3

Left node branch:

- $x_1 = 0, x_2 \leq 2$ (infeasible region) $\rightarrow$ no further branching.

Right node branch:

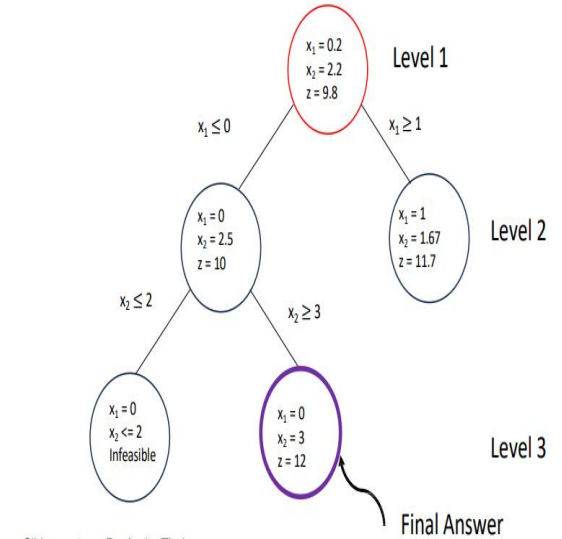
- $x_1 = 0, x_2 = 3, z=12$.
- Both x_1, x_2 are integer $\rightarrow$ stop branching.



Step 6: Should We Branch Right Node at Level 2?

No, because further branching would yield $z \geq 12$, worse than current solutions.

Prune this node to reduce computations.



- All the examples discussed above have 2 design variables, x_1 , and x_2 .
- Branch and Bound Method is applicable for any number of variables.
- We discussed 2 variable problems, because such problems can be solved within a limited amount of time using pen and paper.
- We also carefully chose problems, which are linear and have global optimum.

Problems to practice

Example 4: Maximize $z = -x_1 + 4x_2$ such that $-10x_1 + 20x_2 \leq 22$, $5x_1 + 10x_2 \leq 49$, and $x_1 \leq 10$ where x , the design variables, can only take positive integer values. Determine x , c , A , b . Solve $x = [x_1, x_2]$. The design variables are x_1 and x_2 .

[Solution: $(X_1, X_2) = (2, 2)$, $z = 6$]

Example 5: Maximize $z = x_1 + 4x_2$ such that $2x_1 + 4x_2 \leq 7$, $5x_1 + 3x_2 \leq 15$, where x , the design variables, can only take positive integer values. • Determine x , c , A , b . Solve $x = [x_1, x_2]$. The design variables are x_1 and x_2 .

[Solution: $(X_1, X_2) = (1, 1)$, $z = 5$]

Example 6: Maximize $z = 5x_1 + 6x_2$ such that $x_1 + x_2 \leq 5$ and, $x_1 + 7x_2 \leq 28$ where x , the design variables, can only take positive integer values. Determine x , c , A , b . Solve $x = [x_1, x_2]$. The design variables are x_1 and x_2 .

[Solution: $(X_1, X_2) = (2, 3)$, $z = 28$]

Example 7: Maximize $z = 5x_1 + 8x_2$ such that $5x_1 + 9x_2 \leq 45$ and, $x_1 + x_2 \leq 6$ where x , the design variables, can only take positive integer values. Determine x , c , A , b . Solve $x = [x_1, x_2]$. The design variables are x_1 and x_2 .

[Solution: $(X_1, X_2) = (0, 5)$, $z = 40$]

1. Engineering Optimization Theory and Practice, Fifth Edition, Singiresu S. Rao, 2020.



THANK YOU

Chandravva Hebbi

Computer Science and Engineering



PES
UNIVERSITY

CELEBRATING 50 YEARS

Mathematics for Computer Science Engineers

Unit - 4 Introduction to Nonlinear Optimization

Chandravva Hebbi

Computer Science and Engineering

Mathematics for Computer Science Engineers

Introduction to Nonlinear Optimization



Definition:

Non-linear optimization refers to maximizing or minimizing a non-linear objective function subject to a set of constraints.

Unlike linear optimization (where objective and constraints are linear combinations of decision variables), non-linear optimization deals with complex relationships such as polynomial, exponential, logarithmic, and other nonlinear functions.

Nonlinear Programming (NLP):

Optimization problems with nonlinearities are nonlinear programming (NLP) problems.

Many NLPs are unconstrained (no constraints).

Solutions to NLPs often utilize search procedures.

NLP solutions are harder to determine than LPs, partly due to difficulty distinguishing between local and global minima or maxima.

Solving LP and NLP Problems:

- If an LP problem is feasible, the solution is a "corner point" where constraints intersect, found using the simplex algorithm.
- For many real-world problems, the objective function or constraints may not be linear.

Mathematics for Computer Science Engineers

Introduction to Nonlinear Optimization



Key Characteristics:

- **Objective Function:** Can be quadratic, exponential, logarithmic, trigonometric, etc.
- **Constraints:** May include nonlinear constraints (equalities/inequalities) limiting decision variables.
- **Decision Variables:** Can be continuous or discrete; their relationships can complicate finding optimal solutions.

Importance of Nonlinear Optimization

1) Performance Optimization:

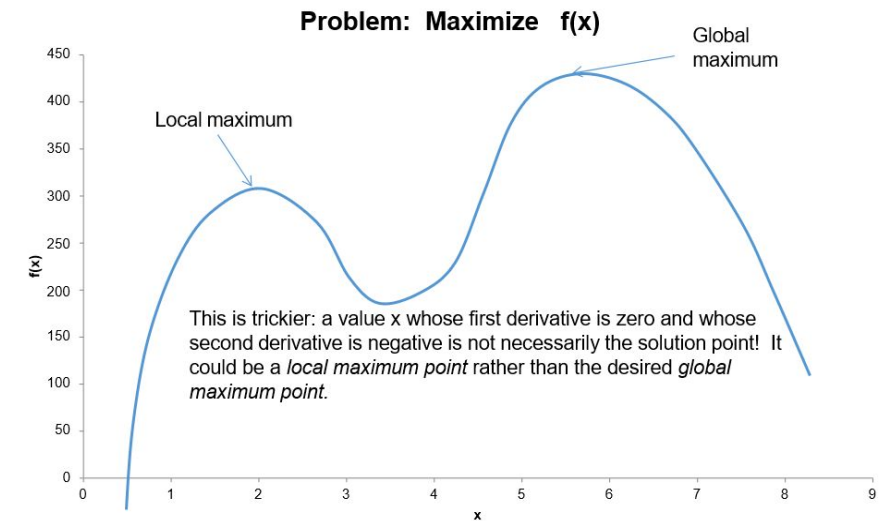
Nonlinear optimization techniques improve performance metrics by:

Reducing Costs: Identifying cost-effective strategies and operational efficiencies.

Increasing Productivity: Streamlining processes to achieve higher outputs with the same or fewer inputs.

2) Decision Making:

Nonlinear optimization helps decision-makers evaluate trade-offs between conflicting objectives, such as cost vs. quality or risk vs. return, aiding organizations in informed strategic decisions.



Usage of Nonlinear Optimization

In machine learning, especially for complex models such as neural networks, the loss functions are typically nonlinear.

Nonlinear optimization methods are essential for:

1) Hyperparameter Tuning:

Finding optimal hyperparameters that affect model performance.

2) Feature Selection:

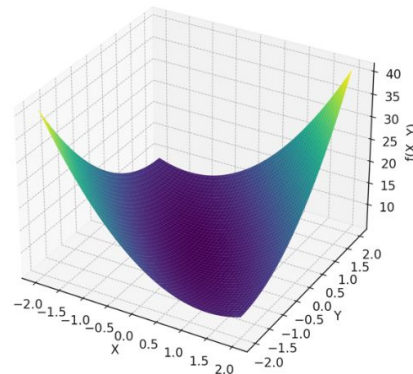
Identifying the most relevant features to improve model accuracy.

3) Neural Network Training:

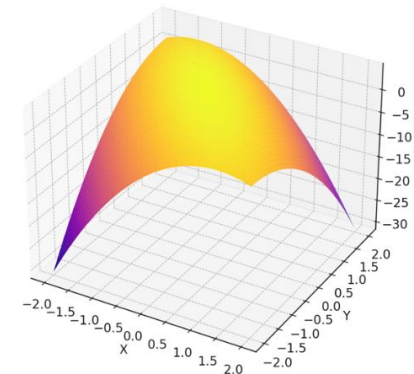
Using algorithms like backpropagation, which rely on gradient descent to **minimize nonlinear loss functions**.

Convex v/s Concave functions

Convex Function: $f(x, y) = 3x^2 + 2y^2 + 4xy + 5$



Concave Function: $g(x, y) = -3x^2 - 2y^2 - 4xy + 5$

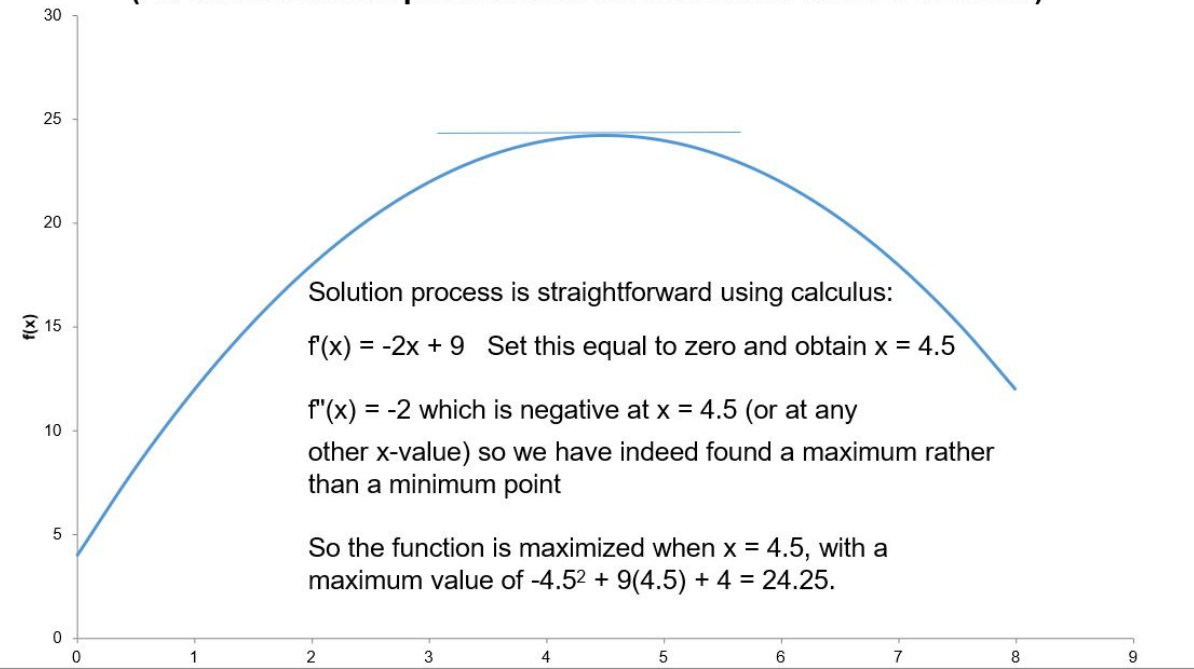


Mathematics for Computer Science Engineers

Unconstrained Nonlinear Example

- **Example problem:** Maximize $f(x) = -x^2 + 9x + 4$
- **Solution process:**
 - Find critical points using calculus:
 - $f'(x) = -2x + 9$, set to zero $\rightarrow x = 4.5$
 - $f''(x) = -2$ (negative at $x=4.5$), confirming a maximum.
- **Maximum value:**
 $f(4.5) = -4.5^2 + 9 * 4.5 + 4 = 24.25$

Example problem: Maximize $f(x) = -x^2 + 9x + 4$
(An unconstrained problem that can be solved without a search)



Gradient Descent

What is Gradient Descent?

Gradient descent is an **iterative optimization algorithm** for finding the minimum of a function, suitable for unconstrained nonlinear optimization problems.

The algorithm takes steps proportional to the negative of the gradient (first derivative) of the function at the current point.

Algorithm Steps:

Initialize the starting point x_0 .

Update the point iteratively:

$$x_{n+1} = x_n - \alpha \nabla f(x_n)$$

Repeat until convergence.

Where:

α : learning rate

$\nabla f(x_n)$: gradient of function at x_n .

Intuitive Example:

- Imagine a **hiker descending a mountain blindfolded**:
 - The hiker knows if she is **going down** (making progress) or **going up** (losing progress).
 - Repeating downhill steps leads to the base.
- Similarly, in machine learning, weights are adjusted based on the loss function until reaching the minimum.

Parameter Update Formula: $w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$

- Update in the **negative gradient direction** to minimize loss.
- Used for loss functions like: Mean Squared Error, Cross-Entropy, Mean Absolute Percentage Error

Learning Rate Effects:

- The **learning rate** determines update size:
 - **Too low**: Many updates are needed before reaching minimum.
 - **Optimal**: Swiftly reaches the minimum point.
 - **Too high**: Drastic updates can lead to divergent behaviour (not reaching minimum).
- **Note**: The optimal learning rate is typically unknown and must be tuned.

Mathematics for Computer Science Engineers

Gradient Descent – Problems



Find the local minima of the function $y = (x+5)^2$ starting from the point $x=3$.

Graphical Solution:

By examining the graph of $y=(x+5)^2$, the function reaches its minimum when $x = -5$.

Therefore, $x = -5$ is the **local and global minimum** of the function

Numerical Solution Using Gradient Descent:

- **Starting Point:** $x_0 = 3$
- **Learning Rate:** 0.01
- The gradient:

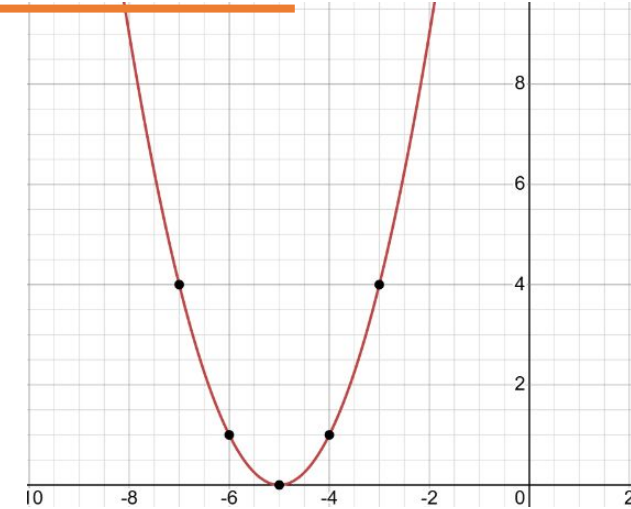
$$\frac{dy}{dx} = \frac{d(x+5)^2}{dx} = 2 * (x + 5)$$

- Update rule:

$$x_1 = x_0 - (\text{learning rate}) * \frac{dy}{dx}$$

- For the first step:

$$x_1 = 3 - (0.01) * 2 * (3 + 5) = 2.84$$



```
# Gradient of the function f(x) = (x+5)^2 -> df/dx = 2(x+5)
df = lambda x: 2 * (x + 5)

# Hyperparameters
cur_x = 3          # Starting point
rate = 0.01        # Learning rate
precision = 0.00001 # Convergence threshold
max_iters = 10000  # Maximum iterations

# Variables for loop
previous_step_size = 1
iters = 0

# Gradient Descent loop
while previous_step_size > precision and iters < max_iters:
    prev_x = cur_x
    cur_x = cur_x - rate * df(prev_x) # Update step
    previous_step_size = abs(cur_x - prev_x) # Step size
    iters += 1

print("The local minimum occurs at:", cur_x)
print("Number of iterations:", iters)
```

Mathematics for Computer Science Engineers

Unbounded NLP Problem

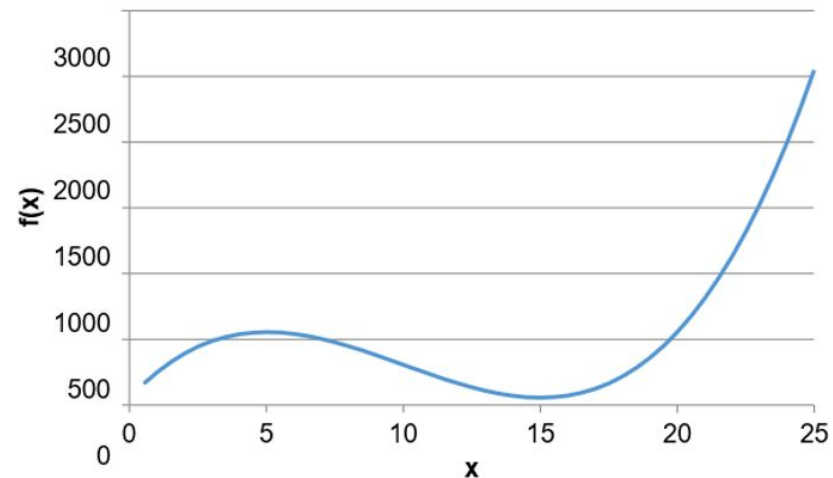
Maximize: $f(x) = x^3 - 30x^2 + 225x + 50$

The problem is unbounded; as x increases, $f(x)$ increases without limit.

Conclusion: There is no finite solution point.

But within an interval, eg. $X=[0,15]$, the maximum point would be at $x=5$.

Maximize $f(x) = x^3 - 30x^2 + 225x + 50$



The correct answer is that the problem is *unbounded*.
There is no solution point!

Observations

1. Behaviour at Infinity

The term with the highest degree (x^3) dominates the function's behaviour as x approaches infinity or negative infinity.

If the coefficient of x^3 is positive, $f(x)$ increases to positive infinity as $x \rightarrow +\infty$ and decreases to negative infinity as $x \rightarrow -\infty$.

2. Local Extrema vs Global Extrema

Critical points found at $x=5$ and $x=15$:

- Local maximum at $x=5$

- Local minimum at $x=15$

These are only local extrema, representing maximum or minimum values within a specific region, not for the entire domain of x .

3. Unbounded Nature of Cubic Functions

Unlike quadratic functions (which can have a global maximum or minimum), cubic functions typically do **not** have global extrema because they extend to infinity in both directions.

Example:

$$f(x) = x^3 - 30x^2 + 225x + 50$$

This function is unbounded; there is no highest or lowest value for $f(x)$ over all x , only local highs/lows.

Mathematics for Computer Science Engineers

Constrained Nonlinear Example

Applications:

- **Resource Allocation Problem:**
Maximize utility from allocating resources across different projects, subject to budget constraints.
- **Support Vector Machine (SVM) Training:**
Train an SVM by maximizing margin between two classes, ensuring points are classified correctly.

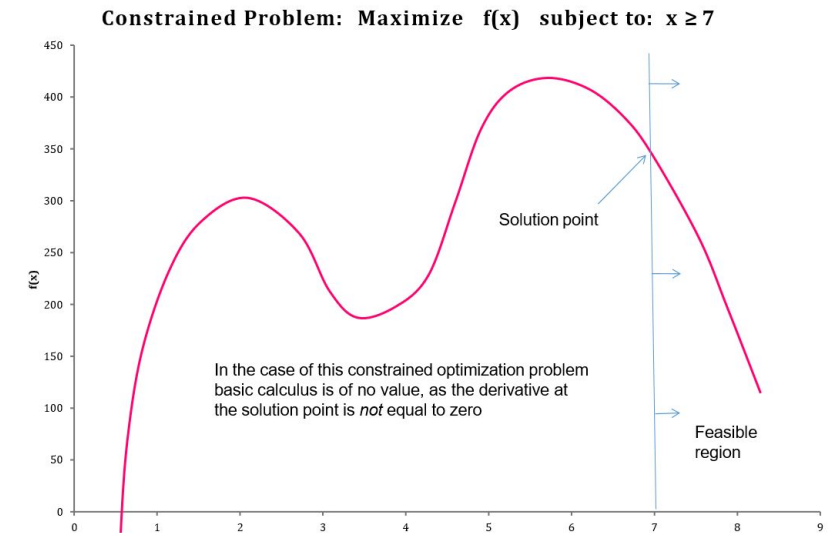
Example:

Maximize $f(x)$ subject to $x \geq 7$

- The solution point may not have a derivative of zero.
- Feasible region: Defined by constraint ($x \geq 7$), and basic calculus might not locate the solution since the optimal point could occur at the boundary.

Methods to Solve Constrained Nonlinear Optimization Problems

- Lagrange Multipliers Method
- Karush-Kuhn-Tucker (KKT) Conditions
- Genetic Algorithms



- The **Lagrange multipliers method** is a classical approach used for problems with equality constraints.
- It transforms a **constrained optimization problem** into an **unconstrained problem** by incorporating the constraints into the objective function.

Lagrangian Function Definition

$$L(x, \lambda) = f(x) + \lambda * g(x)$$

- $f(x)$: objective function
- $g(x) = 0$: constraint
- λ : Lagrange multiplier

Steps:

- Take partial derivatives of the Lagrangian with respect to x and λ , set them to zero, solve the resulting equations.

Applicability:

- Best for problems with equality constraints and differentiable functions.

Mathematics for Computer Science Engineers

Lagrange Multipliers Examples



Example 1: Minimizing a Cost Function with Resource Constraints

• **Objective Function:** $C(x, y) = 4x^2 + 3y^2$

• **Constraint:** $g(x, y) = x + 2y - 10 = 0$

Step 1: Form the Lagrangian Function: $L(x, y, \lambda) = 4x^2 + 3y^2 + \lambda(10 - x - 2y)$

Step 2: Find Partial Derivatives and Set Them to Zero

1. With respect to x :

$$\partial L / \partial x = 8x - \lambda = 0 \implies \lambda = 8x$$

2. With respect to y :

$$\partial L / \partial y = 6y - 2\lambda = 0 \implies \lambda = 3y$$

3. With respect to λ :

$$\partial L / \partial \lambda = 10 - x - 2y = 0 \implies x + 2y = 10$$

Mathematics for Computer Science Engineers

Lagrange Multipliers Examples

Step 3: Substitute Equations

- Equate $8x$ and $3y$:

$$8x = 3y \Rightarrow y = \frac{8x}{3}$$

- Substitute into constraint:

$$x + 2\left(\frac{8x}{3}\right) = 10 \Rightarrow x + \frac{16x}{3} = 10 \Rightarrow \frac{19x}{3} = 10 \Rightarrow x = \frac{30}{19} \Rightarrow x \approx 1.58$$

- Find y :

$$y = \frac{8x}{3} = \frac{8 \cdot 1.58}{3} \approx 4.23$$

Conclusion:

Optimal solution is $x \approx 1.58$, $y \approx 4.23$.

Mathematics for Computer Science Engineers

Lagrange Multipliers Examples



Example 2: Minimize cost for a container with volume $V=1000$,
Where $C(x, h) = 5x^2 + 12xh$ and constraint $x^2h=1000$.

Step 1: Lagrangian Function

$$L(x, h, \lambda) = C(x, h) - \lambda(x^2h - 1000)$$

Substitute cost and constraint:

$$L(x, h, \lambda) = 5x^2 + 12xh - \lambda(x^2h - 1000)$$

Step 2: Compute Partial Derivatives

1. With respect to x :

$$\partial L / \partial x = 10x + 12h - \lambda(2xh) = 0$$

2. With respect to h :

$$\partial L / \partial h = 12x - \lambda x^2 = 0$$

3. With respect to λ :

$$\partial L / \partial \lambda = -(x^2h - 1000) = 0$$

$$\Rightarrow x^2h = 1000$$

4. Substitute λ into Equation:

Substitute $\lambda = 12/x$ into the partial derivative equation:

$$10x + 12h + (-12/x) * (2xh) = 0$$

· Thus,

$$x = 6h/5$$

Substitute x into Constraint to Solve for h:

· Substitute $x=6h/5$ in $x^2h = 1000$:

$$(6h/5)^2h = 1000$$

$$36h^3/25 = 1000$$

$$h \approx 8.88$$

Find x using h:

· Substitute $h \approx 8.88$ into $x = 6h/5$:

$$x = 6*8.88/5 \approx 10.656$$

Find λ :

· Use earlier result $\lambda=12/x$:

$$\lambda = 12/10.656 \approx 1.126$$

Mathematics for Computer Science Engineers

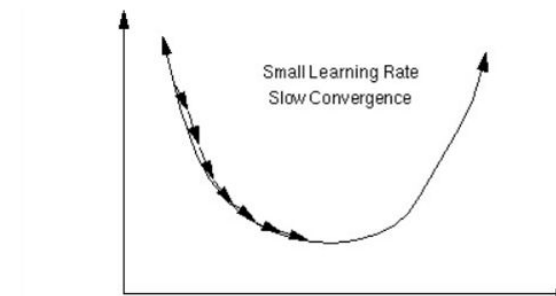
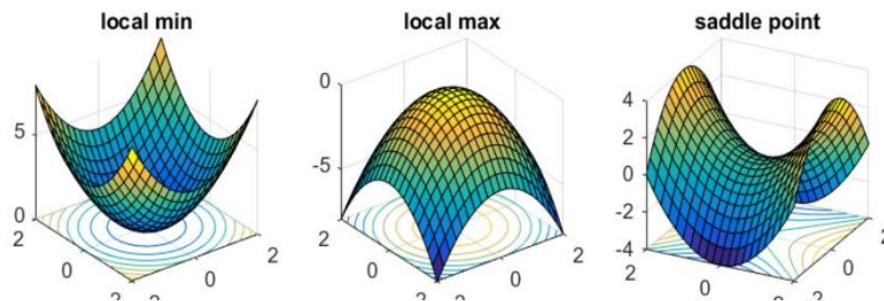
Challenges in Non-Linear Optimization

Main Challenges:

- Nonlinear optimization faces several challenges including **local minima**, **saddle points**, and **slow convergence**.
- Navigating complex objective functions with **multiple local minima** or saddle points can be challenging and may require advanced techniques to find global optima.

Visual Representation:

- Local min:** Bowl-shaped depression (not global minimum)
- Local max:** Peak-shaped elevation (not global maximum)
- Saddle point:** Point where function curves up in one direction and down in another
- Small Learning Rate/Slow Convergence:** Optimization takes many iterations to reach minimum



Definition:

- Convex optimization is a special case of optimization where the objective function is convex, and the feasible region defined by constraints is also convex.
- A function is convex if its second derivative is non-negative (or in the multivariable case, if its Hessian matrix is positive semidefinite).

Key Advantage:

- Convex optimization deals with minimizing or maximizing convex functions over convex sets, which ensures that any local minimum is a global minimum. These problems are fundamental in various fields, including machine learning, control systems, and finance.

What is Convexity?

Before understanding convex optimization, it's essential to understand the concept of convexity:

Convex Function:

- A function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is called **convex** if, for any two points x and y in its domain, the following condition holds:

$$f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y) \text{ for all } \lambda \in [0,1]$$

Mathematics for Computer Science Engineers

Introduction to convex optimization



Convex Set:

- A set S is convex if, for any two points $x, y \in S$, the line segment connecting x and y is also entirely contained within S : $\lambda x + (1-\lambda) y \in S$ for all $\lambda \in [0,1]$

Mathematical Formulation

A typical convex optimization problem can be formulated as:

Minimize $f(x)$

Subject to:

- $g_i(x) \leq 0$ for $i = 1, \dots, m$
- $h_j(x) = 0$ for $j=1, \dots, p$

Where:

- $f(x)$ is a convex function
- $g_i(x)$ are convex inequality constraints
- $h_j(x)$ are affine equality constraints

If the **objective function is convex** and the **feasible region is a convex set**, then **any local minimum is also a global minimum**.

Beyond Lagrange Multipliers

- The Lagrange multiplier method is a powerful technique used to solve constrained optimization problems, particularly in the context of convex optimization.
- It helps find the local maxima and minima of a function subject to equality constraints.

Note: When dealing with **inequality constraints**, how do we solve?

Answer: Use **KKT Conditions**

Karush-Kuhn-Tucker (KKT) Conditions

The Four Conditions:

1. **Stationarity:** The gradient of the Lagrangian must vanish.
2. **Primal feasibility:** The constraints must be satisfied.
3. **Dual feasibility:** The multipliers for inequality constraints must be non-negative.
4. **Complementary slackness:** The product of each inequality constraint and its corresponding multiplier must equal zero.

Mathematics for Computer Science Engineers

Mathematical Formulation of KKT Conditions



Given the optimization problem, the KKT conditions are:

1. Stationarity Condition:

The gradient of the Lagrangian $L(x, \lambda, v)$ must vanish: $\nabla L(x, \lambda, v) = \nabla f(x) + \sum_{j=1}^m \lambda_j \nabla g_j(x) + \sum_{i=1}^p v_i \nabla h_i(x) = 0$

Where:

- $L(x, \lambda, v)$ is the Lagrangian function
- λ_j are the Lagrange multipliers for the equality constraints
- v_i are the Lagrange multipliers for the inequality constraints

2. Primal Feasibility: The original constraints must be satisfied: $g_j(x) = 0$ for $j=1, 2, \dots, m$ and $h_i(x) \leq 0$ for $i = 1, 2, \dots, p$

3. Dual Feasibility: The multipliers for the inequality constraints must be non-negative: $v_i \geq 0$ for $i=1, 2, \dots, p$

4. Complementary Slackness: For each inequality constraint, the product of the multiplier and the constraint must equal zero:

$$v_i h_i(x) = 0 \text{ for } i=1, 2, \dots, p$$

Interpretation:

If a constraint is **active** (i.e., $h_i(x) = 0$), then the corresponding multiplier v_i can be positive.

If the constraint is **inactive** (i.e., $h_i(x) < 0$), then the multiplier must be zero

Case	Type	Constraint	Lagrangian	λ Sign
1	Min	$\leq$	$L = f + \lambda g$	$\lambda \geq 0$
2	Min	$\geq$	$L = f - \lambda g$	$\lambda \leq 0$
3	Max	$\leq$	$L = f - \lambda g$	$\lambda \geq 0$
4	Max	$\geq$	$L = f + \lambda g$	$\lambda \leq 0$

Rule of Thumb

- For **Minimization** $\rightarrow$ use $+\lambda g$
- For **Maximization** $\rightarrow$ use $-\lambda g$
- **Flip the sign** when the constraint is of type “ $\geq$ ”

KKT Example

Example 1

Minimize: $f(x) = x^2$ **Subject to:** $g(x) = 2 - x \leq 0$

Step 1: Write Down the KKT Conditions

1. **Stationarity:** The derivative of the Lagrangian with respect to x must be zero.
2. **Primal feasibility:** The constraint $g(x) \leq 0$ must be satisfied.
3. **Dual feasibility:** The Lagrange multiplier $\lambda \geq 0$.
4. **Complementary slackness:** $\lambda g(x) = 0$.

The Lagrangian is: $L(x, \lambda) = x^2 + \lambda(2 - x) = 0$

Where $\lambda \geq 0$ is the Lagrange multiplier associated with the constraint.

Step 2: Set Up the Stationarity Condition

The stationarity condition requires that the derivative of the Lagrangian with respect to x is zero: $\partial L / \partial x = 2x - \lambda = 0$

From this, we have: $\lambda = 2x$

Step 3: Apply the Primal Feasibility Condition

The constraint $g(x) = 2 - x \leq 0$ must hold, which implies: $x \geq 2$

KKT Example

Step 4: Apply the Complementary Slackness Condition

The complementary slackness condition states that $\lambda g(x) = 0$.

This means we have two cases to consider:

Case 1: $\lambda=0$

- If $\lambda=0$, then from the stationarity condition: $2x - \lambda = 0 \rightarrow x = 0$
- However, $x=0$ does not satisfy the constraint $x \geq 2$ from primal feasibility condition.
- So, this case is not feasible.

Case 2: $g(x)=0$ (i.e., the constraint is active)

- If $2-x = 0$, then $x=2$.
- Substituting $x=2$ into $\lambda = 2x$: $\lambda = 2 * 2 = 4$

Final Solution

Since Case 1 is not feasible, the optimal solution is from Case 2:

- **Optimal point:** $x = 2$
- **Lagrange multiplier:** $\lambda = 4$
- **Minimum value:** $f(2) = 4$

KKT Example

Example 2

To maximize the function $z=2x_1^2 - 7x_2^2 + 12x_1x_2$ subject to the constraint $2x_1 + 5x_2 \leq 98$, we can use the method of Lagrange multipliers.

Step 1: Set Up the Lagrangian Function

To solve this constrained optimization problem, we first set up the **Lagrangian function**:

$$L(x_1, x_2, \lambda) = 2x_1^2 - 7x_2^2 + 12x_1x_2 - \lambda(-98 + 2x_1 + 5x_2)$$

Since the constraint is an inequality, we will approach it by initially treating it as an equality and examine the case where $\lambda \geq 0$ to ensure the constraint is respected.

Step 2: Find Critical Points Using Partial Derivatives

To find the critical points, we need to set the partial derivatives of $L(x_1, x_2, \lambda)$ with respect to x_1 , x_2 , and λ equal to zero.

1. Partial derivative with respect to x_1 :

$$\partial L / \partial x_1 = 4x_1 + 12x_2 - 2\lambda = 0 \quad (1)$$

2. Partial derivative with respect to x_2 :

$$\partial L / \partial x_2 = -14x_2 + 12x_1 - 5\lambda = 0 \quad (2)$$

3. Partial derivative with respect to λ : $\partial L / \partial \lambda = 98 - 2x_1 - 5x_2 = 0$ (3)

Step 3: System of Equations and Complementary Slackness

This gives us the system of equations:

1. $4x_1 + 12x_2 - 2\lambda = 0$
2. $12x_1 - 14x_2 - 5\lambda = 0$
3. $98 - 2x_1 - 5x_2 = 0$

Additionally, we have the **complementary slackness condition**: $\lambda g(x) = 0$

Where $g(x) = 98 - 2x_1 - 5x_2 \geq 0$ is the constraint, and $\lambda \geq 0$.

This condition implies that either:

1. $\lambda = 0$, which would mean the constraint is **inactive** (not binding) at the optimum, or
2. $g(x) = 0$, which means the constraint is **active** (binding) at the optimum, and $\lambda > 0$.

Let's consider these two cases separately to find the maximum.

Case 1: $\lambda = 0$ (Constraint Inactive)

Substitute $\lambda = 0$ in equations (1) and (2):

$$4x_1 + 12x_2 = 0$$

$$12x_1 - 14x_2 = 0$$

This gives us:

$$x_1 = 0, x_2 = 0, \text{ and } z = 0$$

Result:

The solution is trivial, yielding a point at the origin.

Case 2: $\lambda \neq 0$ (Constraint Active)

Solving the three linear equations, we obtain:

$$x_1 = 44$$

$$x_2 = 2$$

$$\lambda = 100$$

Maximum Value:

$$Z_{\max} = 4900$$

1. Minimize $f(x,y)=x^2+y^2$ subject to $x + y \geq 1$. Solution: $(x,y)=(\frac{1}{2},\frac{1}{2})$
2. Minimize $f(x,y)=x^2+4y^2$ subject to $x + 2y = 6$ Solution: $(x,y)=(3,3/2)$
3. Minimize $f(x,y)=(x-1)^2+(y-2)^2$ subject to $x + y \leq 1$ Solution: $(x,y)=(1,0)$
4. Maximize $f(x,y)=xy$ subjected to $x+y=10, x \geq 0, y \geq 0$. Solution: $(x,y)=(5,5)$
5. Minimize $f(x,y)=x^2+2y^2$ subject to $x + y \geq 3, x \geq 0$. Solution: $(x,y)=(2,1)$

Mathematics for Computer Science Engineers

References



1. Engineering Optimization Theory and Practice, Fifth Edition, Singiresu S. Rao, 2020



THANK YOU

Chandravva Hebbi

Computer Science and Engineering

Lagrange Multiplier Technique

- The Lagrange multiplier technique lets you find the maximum or minimum of a multivariable function $f(x, y, \dots)$ when there is some constraint on the input values you are allowed to use.
- This technique only applies to constraints that look something like this:

$$g(x, y, \dots) = c$$

Here, g is another multivariable function with the same input space as f , and c is some constant.

- The core idea is to look for points where the contour lines of f and g are tangent to each other.
- This is the same as finding points where the gradient vectors of f and g are parallel to each other.
- The entire process can be boiled down into setting the gradient of a certain function, called the **Lagrangian**, equal to the zero vector.

Procedure of Lagrange Multiplier Technique

- **Step 1:** Introduce a new variable λ , and define a new function L as follows:

$$L(x, y, \dots, \lambda) = f(x, y, \dots) - \lambda (g(x, y, \dots) - c)$$

Or

$$L(x, y, \dots, \lambda) = f(x, y, \dots) + \lambda (c - g(x, y, \dots))$$

This function L is called the "Lagrangian", and the new variable λ is referred to as a "Lagrange multiplier"

- **Step 2:** Set the gradient of L equal to the zero vector.

$$\nabla \mathcal{L}(x, y, \dots, \lambda) = \mathbf{0} \quad \leftarrow \text{Zero vector}$$

In other words, find the **critical points** of L .

- **Step 3:** Consider each solution, which will look something like $(x_0, y_0, \dots, \lambda_0)$.

Plug each one into f . Or rather, first remove the λ_0 component, then plug it into f , since f does not have λ as an input. Whichever one gives the greatest (or smallest) value is the maximum (or minimum) point you are seeking.



PES
UNIVERSITY

CELEBRATING 50 YEARS

Mathematics for Computer Science Engineers

Unit - 4 Genetic and Evolutionary Optimization

Chandravva Hebbi

Computer Science and Engineering

However, many *real-world problems* do not satisfy these mathematical assumptions.

The search space may be **discontinuous, noisy, nonlinear, or multi-modal** (many peaks and valleys).

In such cases, traditional methods like Gradient Descent or Newton's Method often get **stuck in local minima** or even **fail to** converge.

The traditional methods require:

- The function to be continuous and differentiable.
- Starting points close to the global optimum.
- A well-behaved function (no many peaks or discontinuities).

To handle such complexity, we need **optimization techniques** that can:

- Work without gradient or derivative information.
- Explore a large, complex search space efficiently.
- Adapt to dynamic or uncertain environments.

Mathematics for Computer Science Engineers

Examples

The Problem: Real-world functions are messy or too complex as given below

Challenge	Example	Why classical methods fail
Nonlinear	$f(x) = x \sin(10\pi x)$	Many local optima $\rightarrow$ gradient methods get stuck
Discrete / combinatorial	Traveling salesman, scheduling	No derivatives possible
Multi-modal	Neural net hyperparameter tuning	Many peaks $\rightarrow$ local minima traps

This need gave rise to a new class of methods collectively known as **Computational Intelligence (CI)** or **Soft Computing** techniques

Computational Intelligence Techniques

Computational Intelligence refers to a set of adaptive problem-solving techniques that mimic human reasoning and learning to handle complex, imprecise, and uncertain problems where traditional mathematical models fail.

Key Characteristics:

- Works with imprecision, approximation, and partial truth.
- Learns and adapts from data or experience.
- Often bio-inspired, but not necessarily based on nature.

Major Algorithms of CI:

Artificial Neural Networks (ANNs) – inspired by the brain's learning.

Fuzzy Logic Systems – inspired by human reasoning with uncertainty.

Evolutionary Computation (like Genetic Algorithms) – inspired by biological evolution.

Swarm Intelligence (e.g., PSO, ACO) – inspired by social behavior of animals.

Goal: Create intelligent systems capable of **learning, adaptation, and decision making**

Mathematics for Computer Science Engineers

Nature-Inspired Computing



Nature-Inspired Computing draws inspiration directly from biological, physical, chemical, or ecological processes found in nature to design algorithms and computational systems.

Key Idea:

It models how natural systems solve problems — like ant colonies finding shortest paths, birds flocking, or fireflies communicating — and applies those mechanisms to computational optimization.

Common Nature-Inspired Algorithms:

- **Genetic Algorithm (GA)** – biological evolution
- **Particle Swarm Optimization (PSO)** – bird flocking
- **Ant Colony Optimization (ACO)** – ant foraging behavior(the process by which ants search their food)
- **Artificial Bee Colony (ABC)** – bee food foraging
- **Firefly Algorithm, Cuckoo Search, Bat Algorithm**, etc.

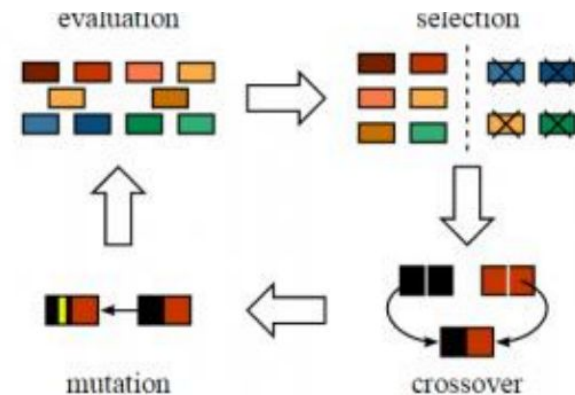
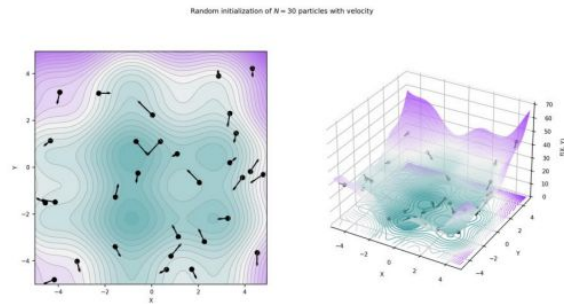
Goal: Use **natural phenomena** as metaphors to develop **efficient optimization methods**.

Note: All Nature-Inspired Computing techniques are part of Computational Intelligence, but not all Computational Intelligence techniques are nature-inspired.

Bio-Inspired Computing (BIC) is a subfield of artificial intelligence and optimization that takes **inspiration from biological systems and processes** found in nature — such as **evolution, cell growth, swarm behavior, and immune systems** — to design algorithms and computational models that can solve complex problems.

Examples of Algorithms:

- Genetic Algorithm (Evolution)
- Particle Swarm Optimization (Swarm behavior)
- Ant Colony Optimization (Ant foraging)
- Artificial Immune System (Immune defense)



Bio-Inspired Algorithms mimic *adaptation, cooperation, and learning* in nature to explore the solution space efficiently and escape local optima.

GA belongs to the family of evolutionary algorithms, inspired by natural selection. It does not need derivatives, does not assume continuity, and works with complex or discrete search spaces. Instead of following the slope like gradient descent, it evolves a population of possible solutions over time.

General Principles

- Genetic Algorithms are metaheuristic, search-based optimization techniques that use mechanisms analogous to biological mutation, crossover, and selection.
- The process begins with a population of trial solutions and evolves these to optimize performance over time, referencing the concept of "survival of the fittest" from Darwinian evolution.

Key Features

- GAs start with a population (set) of candidate solutions rather than optimizing a single solution at a time.
- For n decision variables, the recommended population size typically ranges from $2n$ to $4n$. This diversity helps to avoid convergence on local optima.
- GAs relies only on the objective function values, not on additional derivative information. This makes them suitable for complex problems where derivatives are hard to compute or do not exist.

Population

- A population: Set of all possible (encoded) solutions to a given problem, also called the pool of individuals, which enables the application of genetic operators.

Chromosome / Individual

- A chromosome (individual): Represents one solution to the problem and is the carrier of genetic information. The fitness (objective function value) characterizes the solution's quality within the search space.

Gene

- Gene: One element position within a chromosome. Ex: Eye color

Allele

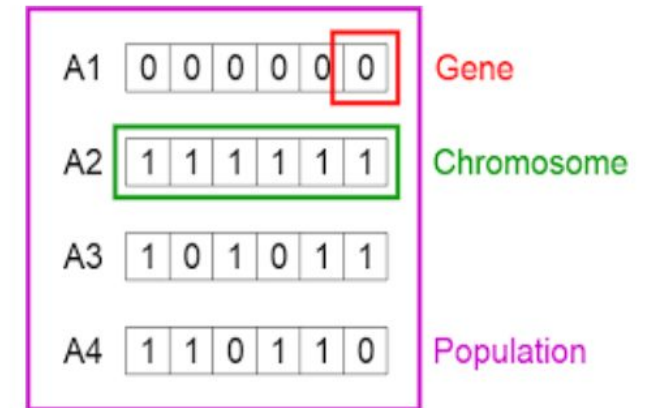
- Allele: The value a gene takes for a particular chromosome. Ex: Eye color=brown, blue

Fitness Function

- The fitness function evaluates the suitability of each solution, outputting a score that indicates "How good a solution is?".
- The fitness function is used synonymously with the objective function in evolutionary computations.

Population and Gene Mapping

Similar diagrams to above, but now highlight the specific role the fitness value (last column) plays for each chromosome.



Mathematics for Computer Science Engineers

Genetic Operators and Evolutionary Algorithm Features

Selection

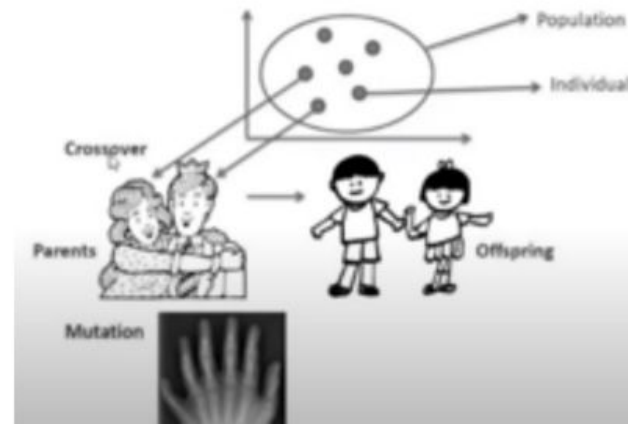
- Selection methods include Roulette Wheel, Tournament, and Steady State selection. The aim is to enable good (fit) chromosomes to participate in creating the next generation by duplicating the best and discarding the worst.

Crossover

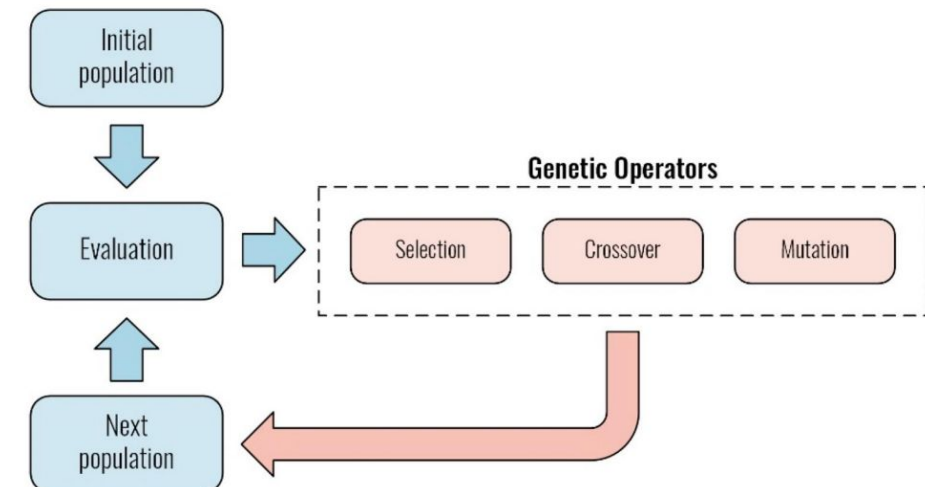
- Types of crossovers include single-point, simulated binary, and linear crossover. Crossover creates new solutions by mixing two (or more) parent chromosomes, promoting global search and the possibility of better solutions.

Mutation

- Mutation involves random or polynomial changes made to genes, ensuring diversity in the population and preventing premature convergence.
- For example, bitwise mutations: $010110 \rightarrow 010100$.



Genetic Operators – These alter the genetic composition of the offspring.



Mathematics for Computer Science Engineers

Genotype, Phenotype, and Generations



Genotype

- Represents the population in computation space, i.e., encoded solutions easily processed by computers.
- **Genotype:** Bb (one brown-eye allele and one blue-eye allele)

Phenotype

- Represents how the encoded (genotype) solution maps to a real-world solution, contextualized for actual problems.
- **Phenotype:** Brown eyes (because brown is dominant over blue)

Generation

- Denotes a single iterative cycle in evolutionary algorithms, marking the progress of solutions through multiple iterations.

Example

P Generation (Parental generation):

- **Parent 1:** BB (brown eyes)
- **Parent 2:** bb (blue eyes)

These are the *original parents* we start with.

F₁ Generation (First filial generation):

- When $BB \times bb \rightarrow$ all offspring are **Bb**

F₁ offspring: All have **brown eyes** (because B is dominant).

Even though they look brown-eyed, they *carry* the blue-eye gene.

Mathematics for Computer Science Engineers

Genotype, Phenotype, and Generations



Basic Structure of a GA

- **Fitness Function:** Evaluates and assigns a fitness value to each candidate solution.
- **Genetic Operators:**
 - Selection (Survivor + Parent): Determines which solutions reproduce.
 - Crossover: Recombines genetic material between solutions.
 - Mutation: Introduces random variations for diversity.
- **Termination Condition:** Defines the stopping criteria for the algorithm.

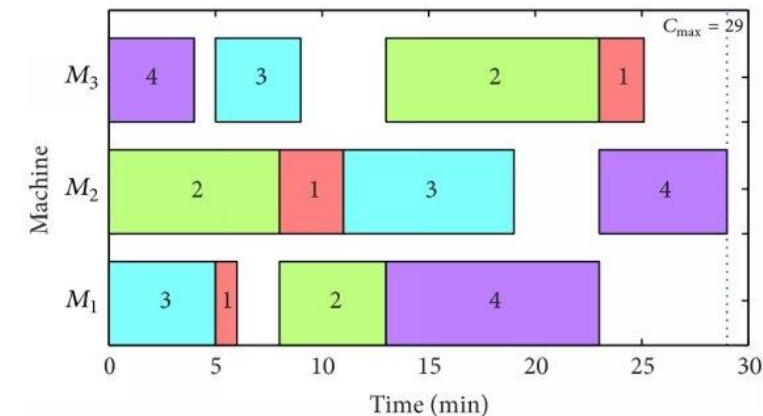
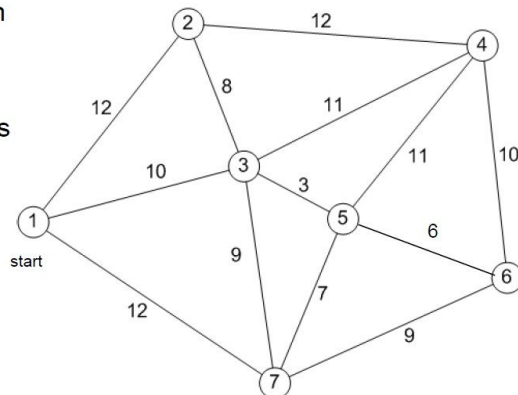
Mathematics for Computer Science Engineers

Applications of Genetic Algorithms

Domain	Optimization Role	GA Use
Engineering Design	Structural shape, antenna, circuit	Nonlinear, multi-variable optimization
Machine Learning	Neural architecture search, feature selection	Non-differentiable cost functions
Scheduling	Job-shop, vehicle routing	Discrete combinatorial optimization
Finance	Portfolio optimization	Discrete and nonlinear constraints

The Traveling Salesman Problem

- The salesman must travel to all cities once before returning home
- The distance between each city is given, and is assumed to be the same in both directions
- Objective - Minimize the total distance to be travelled

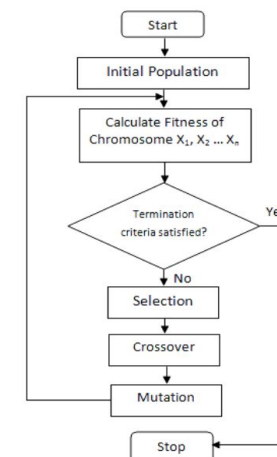


Mathematics for Computer Science Engineers

Genetic Algorithm

1. Start: Initialize a population of N chromosomes (candidate solutions), either randomly or seeded by heuristic methods.
2. Fitness Evaluation: Calculate the fitness of each chromosome, which quantifies how good the solution is.
3. New Population Creation:
 - Selection: Select two chromosomes from the current population using a selection method that favors fitter individuals.
 - Crossover: Perform crossover operations on the selected chromosome pair to create offspring by mixing genetic material.
 - Mutation: Apply mutation operators to introduce small random changes, maintaining population diversity.
4. Replacement: The newly created offspring replace individuals in the existing population.
5. Termination Check: Assess if the termination condition is met (e.g., a target fitness value or maximum generations). If yes, stop; otherwise, continue iterating.

Each cycle through these steps is called a generation.



Genetic Algorithm Optimization Example

Imagine a robotics competition where teams design small delivery robots that travel in straight lines on a grid. Each robot can move forward by a distance x (measured in meters).

The robot's delivery efficiency is measured by how much area it can cover on the grid in one run before stopping. The covered area increases with both speed and travel distance.

After several trials, the judges find that the delivery efficiency of a robot is proportional to the square of the distance it travels.

Hence, if a robot travels x meters forward, the efficiency score is given by: $f(x) = x^2$, where x is the robot's travel distance in meters (and must be an integer between 0 and 31 due to energy limits).

Objective: Maximize $f(x)=x^2$ where x is an integer in the range $0 \leq x \leq 31$.

Binary strings represent possible values of x , (eg. $5 \rightarrow 00101$).



GAs work with chromosomes, we must encode x as a chromosome (a string).

Types of Encoding: **Binary Encoding** → use bits (0s and 1s) and **Real-Value Encoding** → use actual numbers (floats)

Encoding Solution: Binary Representation

Each possible x is represented as a 5-bit binary string because $2^5 = 32$ covers all integers from 0 to 31. For example:

- $x = 5$ is represented as binary 00101
- $x = 25$ is represented as binary 11001

Using binary strings as chromosomes allows the GA to apply standard genetic operators like crossover and mutation.

Step 1: Initialization (This random start gives a diverse gene pool to begin the search.)

- Randomly Generate a population of 4 binary strings randomly.
- Population: {00101, 11001, 00011, 10100}
- Corresponding values: $x = \{5, 25, 3, 20\}$
- Fitness values: $f(x) = \{25, 625, 9, 400\}$

Mathematics for Computer Science Engineers

Genetic Algorithm Optimization Example



Step 2: Selection using Roulette Wheel

- Selection picks individuals based on their relative fitness probabilities.
- Each string (individual) is selected from a population with probability proportional to its fitness:

$$P(i) = \text{Fitness of string } i / \text{Total Fitness} \quad p_i = \frac{F_i}{\sum_{j=1}^n F_j} \quad \text{for all } i=1, 2, \dots, n$$

Where F_i is the fitness of the i^{th} string. Roulette wheel selection is a widely used approach. The wheel is divided into segments proportional to the fitness values. Multiple spins create a mating pool that statistically reflects fitness-based selection probabilities.

Fitness Function Construction

A widely-used transformation for an unconstrained minimization problem is:

- Calculate total fitness: $25 + 625 + 9 + 400 = 1059$

$$F(X) = \frac{1}{1 + f(X)}$$

- Probabilities of selection for each individual:

$$P = \left\{ \frac{25}{1059}, \frac{625}{1059}, \frac{9}{1059}, \frac{400}{1059} \right\}$$

- Higher fitness individuals have a greater chance to be selected for reproduction, preserving strong traits.

Mathematics for Computer Science Engineers

Genetic Algorithm Optimization Example



Step 3: Crossover (Recombination)

- **Creating new strings (solutions)** by exchanging information between selected parent strings in the mating pool.
- **Different Crossover Approaches**
 - a. One Point Crossover
 - b. Two-Point Crossover
- **Mechanism**
 - a. Two parent strings are randomly selected.
 - b. A crossover site is randomly chosen along the length of the strings.
 - c. The portions to the right of the crossover site are swapped between parents to generate two child strings.
- Example parents:

Parent 1 = 11001 ($x = 25$)

Parent 2 = 10100 ($x = 20$)
- Single-point crossover after the 3rd bit:
 - Offspring 1: First 3 bits of Parent 1 + last 2 bits of Parent 2 $\rightarrow$ 11000 (decimal 24)
 - Offspring 2: First 3 bits of Parent 2 + last 2 bits of Parent 1 $\rightarrow$ 10101 (decimal 21)
- This recombines traits from both parents, potentially producing children with better fitness.

Mathematics for Computer Science Engineers

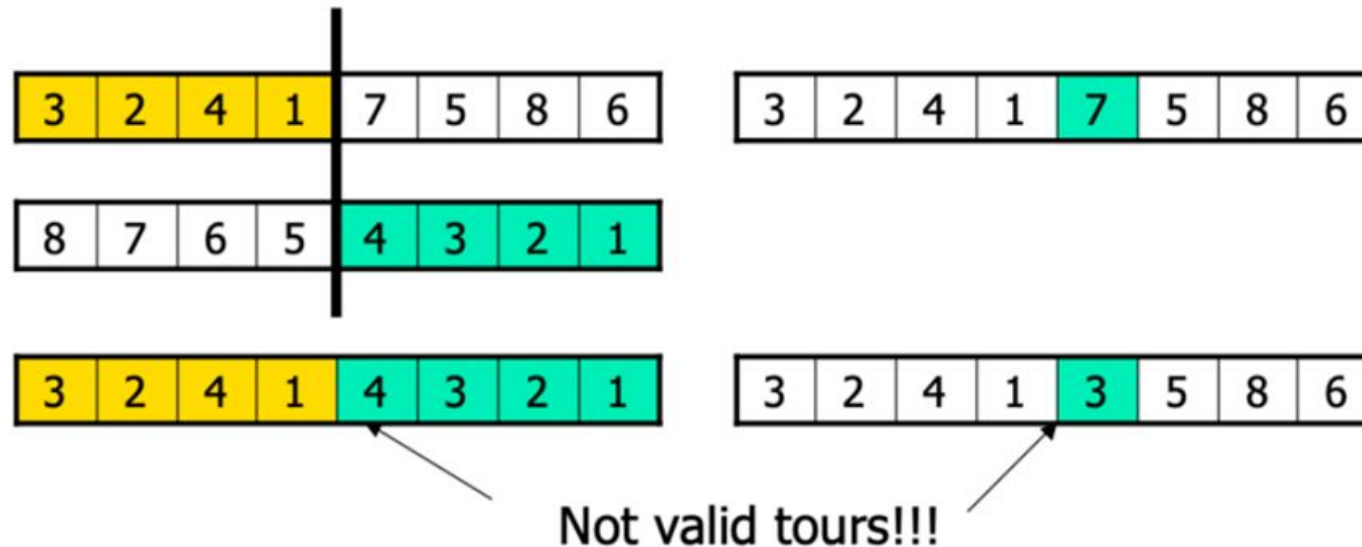
Genetic Algorithm Optimization Example

Problem with single-point crossover

- When solving the Traveling Salesman Problem (TSP), a solution must be a valid permutation of cities.
- Standard crossover can produce **invalid tours** that repeat cities or miss cities.

Example

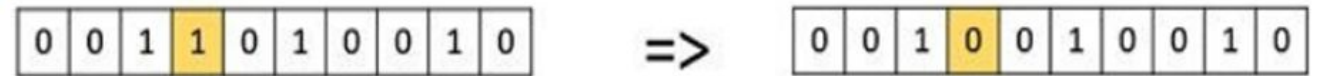
- If single-point crossover is performed, the children may have duplicate cities or not include all required cities, leading to invalid TSP solutions ("Not valid tours!!!").



Genetic Algorithm Optimization Example

Step 4: Mutation

- Mutation is applied after crossover to introduce small, random changes in offspring strings.
- Done with a small mutation probability (P_m), mutation helps avoid stagnation and prevents loss of genetic material (e.g., getting stuck at local minima).
- Mutation randomly changes a binary digit at a selected position (flip $0 \leftrightarrow 1$).
- Applied to offspring resulting from crossover; for example (using single point mutation):
 - Offspring 1: $11000 \rightarrow 11100$ ($x=28$) at position 3.
 - Offspring 2: $10101 \rightarrow 10111$ ($x=23$) at position 4
- The bit flip can be applied to one or both offspring, increasing genetic diversity.
- **Different Mutation Approaches**
 - **Bit Flip:** Invert a selected bit ($0 \leftrightarrow 1$).
 - **1's Complement:** Flip all bits.



Let $C = [01101011]$ be a chromosome, then one's complement operator gives $C' = [10010100]$.

- **Random Resetting / Uniform Mutation:** Swap, scramble, boundary mutation, inversion, insertion, or heuristic mutation.

Step 5: Replacement

- Replace the old population with the new offspring.
 - This can be done either:
 - Partially (elitism): Keep top-performing individuals (e.g., best 2).
 - Fully: Replace all individuals with new offspring.
- Random replacement is possible or elitist strategies may keep top individuals.
- Example new population: {11100, 10111, 00011, 10100}.
- Fitness values re-calculated for new strings: $f(x) = \{784, 529, 9, 400\}$.

Step 6: Termination

- Repeat Steps 2 to 5 for many generations until one of the conditions is met:
 - The maximum number of generations is reached, or
 - Fitness value improvement stagnates.
 - Example: Stop after 50 generations or when the best solution approaches the global minimum of $f(x)$.

Conclusion

- After several generations, the Genetic Algorithm may converge to the optimal solution (e.g., $x = 31$, so $f(31) = 961$).
- This demonstrates the GA's effectiveness in optimizing functions like $f(x)=x^2$

1. Minimize $f(x, y) = x^2 + y^2$
2. Find optimal solution

Suppose we want to minimize

$$f(x) = x^2 + 4 \sin(x)$$

over $-10 \leq x \leq 10$.

3. We are given N items where each item has some weight and profit associated with it. We are also given a bag with capacity W, [i.e., the bag can hold at most W weight in it]. The target is to put the items into the bag such that the sum of profits associated with them is the maximum possible.
4. Find the shortest route for a salesman to visit n cities exactly once and return to the starting point. A salesman needs to visit 5 cities (A, B, C, D, E), and the distance between each pair of cities is given in the following table:

City	A	B	C	D	E
A	0	10	15	20	25
B	10	0	35	25	30
C	15	35	0	30	20
D	20	25	30	0	15
E	25	30	20	15	0

Mathematics for Computer Science Engineers

References



1. Engineering Optimization Theory and Practice, Fifth Edition, Singiresu S. Rao, 2020.



THANK YOU

Chandravva Hebbi

Computer Science and Engineering